

NDNSF: A DATA-CENTRIC SERVICE FRAMEWORK FOR SECURE, COLLABORATIVE,
AND DYNAMIC NETWORKED APPLICATIONS

by

Tianxing Ma

A Dissertation Proposal

Submitted for Committee Review

Academic Advisor:

Dr. Lan Wang

Major: Computer Science

The University of Memphis

September 2026

Abstract

Networked applications need mechanisms for service discovery, invocation authorization, and application-data exchange. Changes in service availability, resources, or connectivity make these tasks more challenging. Named Data Networking (NDN) provides data-centric communication primitives, but applications still need service-level mechanisms. This dissertation investigates these mechanisms through the Named Data Networking Service Framework (NDNSF) and its extension to multi-Provider collaboration.

NDNSF provides reusable service interfaces and Request, acknowledgment (ACK), Selection, and Response Data exchanges. Its initial prototype selects one Provider per invocation: before selection, the Request carries a discovery descriptor readable only by Providers authorized for the requested service; after selection, request-scoped keys deliver the application input only to the selected Provider. NDNSF uses attribute-based encryption (ABE) to protect the discovery descriptor from parties lacking the required decryption capability. The User need not enumerate Provider identities, but authorized candidates can read the descriptor even if they are not selected. Service authorization combines signature validation under trust rules with current service-permission checks. Successful matching of the protected RequestNonce and ProviderChallenge values provides evidence that the parties recovered the values required for the current exchange, while request-state checks reject replayed or stale invocations. Permissions remain separate from identity-signing credentials; runtime grants and withdrawals require validated status and key updates. After selection, request-scoped keys protect application input and results for the intended participants.

The proposed collaboration extension assigns roles to Providers whose positive ACKs pass validation and eligibility checks. Each Provider derives its input-validation rules from the signed plan. The plan binds assigned Providers to service roles and declared dependency edges. These checks constrain which intermediate Data may advance execution, including Data retrieved from caches. The study also examines known-Provider invocation, trusted local composition,

streaming, and retrieval of large objects by name.

Two applications provide validation settings: the unmanned aerial vehicle (UAV) application covers control, telemetry, video, and multi-view analysis; the distributed-inference application covers model retrieval, single-Provider inference, and collaborative execution of model partitions. Preliminary experiments report configuration-specific invocation latency and the number of executions observed under each configuration; they do not yet validate the collaboration extension. Planned evaluation compares complete authorization alternatives, tests dependency enforcement and recovery, and measures completion, latency, communication, computation, and credential-management costs. The expected contribution is an evaluated framework and evidence identifying when its mechanisms benefit dynamic applications and when their overhead limits those benefits.

Keywords: Named Data Networking, service authorization, provider discovery, collaboration, distributed inference

Table of Contents

Chapter	Page
1 Introduction	1
1.1 Application Motivation and Service-Level Decisions	1
1.2 The Initial NDN SF Foundation	2
1.3 Application Needs and Framework Scope	2
1.3.1 Research Gap and Design Focus	4
1.4 Extending Invocation to Multi-Provider Collaboration	6
1.5 Thesis and Research Questions	6
1.6 Proposal Scope and Application Responsibility	8
1.7 Expected Contributions and Current Foundation	8
2 Background and Related Work	10
2.1 NDN Background and Naming Responsibilities	10
2.1.1 Named Retrieval, Forwarding State, and Object Identity	10
2.1.2 From NDN Building Blocks to Service Decisions	11
2.2 Remote Procedure Call (RPC), Robotics Middleware, and Messaging Alternatives	12
2.2.1 RPC Interfaces and Microservice Integration	12
2.2.2 Robot Components and Asynchronous Messaging	13
2.2.3 Microservices and Edge Computing	14
2.3 Named Computation, Invocation, and Composition	15
2.4 Comparing Authorization Designs under the Same Requirements	16
2.4.1 DNMP as a Concrete Trust-Schema Design	16
2.4.2 Adapting the Alternative to NDN SF	16
2.5 Protected Discovery and Permission Management	18
3 NDN SF Framework and Proposed Extensions	19
3.1 Components and Application Responsibilities	19
3.2 Participants, Trust, and Exclusions	21
3.3 Four Named Message Types	22
3.4 Authorization and Confidentiality	24
3.4.1 Rationale for ABE-Backed Service Authorization	24
3.4.2 Authorization Mechanisms, Lifecycle, and Costs	26
3.5 Candidate Information, Selection, and Admission	29
3.6 Invocation Modes and Execution Lifecycle	30
3.6.1 Discovery, Known Providers, and Local Composition	30
3.6.2 Admission, Progress, and Failure	31
3.7 Control Messages, Streams, and Complete Objects	32
3.8 Proposed Extension: From ACKs to a Collaboration Plan	33
3.9 Proposed Dependency Admission and Execution State	34
3.10 Conditional Correctness Argument	36

4	UAV Application over NDNF	37
4.1	Application Structure and Service Workflows	37
4.2	Operator Workflow and State Interpretation	38
4.3	Live Video, Recording, and Perception Services	38
4.4	Flight Commands and Local Safety	40
4.5	Mission Assignment and Partial Failure	41
4.6	Representative Collaboration: Multi-View Collection and Analysis	41
5	Distributed Inference over NDNF	43
5.1	Application Structure and Inference Workflows	43
5.2	Representative Collaboration: Multi-Provider Inference	44
5.3	From a Model Graph to Runtime Assignments	45
5.4	Planning Inputs, Feasibility, and Policy Comparisons	46
5.5	Tensor Exchange and Autoregressive State	47
6	Research and Evaluation Plan	49
6.1	Cross-Application Validation	49
6.2	Preliminary Evidence and Its Limits	51
6.3	Evidence Needed for Each Research Question	53
6.4	Application Experiments and Interpretation	53
6.5	Primary Research Scope and Feasibility	59
6.6	Existing Foundation and Remaining Research Work	59
6.7	Timeline and Decision Points	60
6.8	Risks, Mitigation, and Limits	61
7	Conclusion	62
A	Preliminary Prototype Observations and Evidence Limits	63
	References	64

Chapter 1

Introduction

1.1 Application Motivation and Service-Level Decisions

Networked applications combine services with different lifetimes and resource needs. A UAV operator issues commands to a particular vehicle, follows changing telemetry, and retrieves video or recorded images. An inference application selects compute resources, obtains model artifacts, and exchanges intermediate results. Across these workflows, developers must associate requests, permissions, execution state, and data objects even when connectivity or available resources change.

A requester may know the required service without knowing which authorized Providers are currently reachable, capable, and willing to execute it. Other operations must retain an already selected Provider, such as stopping a particular camera. The framework must therefore support runtime selection while keeping later commands and results associated with the intended Provider and operation. Here, User, Provider, and Controller are logical protocol roles: a User requests a service, a Provider offers or executes it, and a Controller administers service permissions; one node may hold more than one role. The research question is how a reusable service framework can support these decisions and data exchanges without requiring each application to define its own invocation protocol.

Named Data Networking (NDN) supports name-based Data retrieval and data-centric security [1, 2]. Names identify Data, guide Interest forwarding, and participate in trust rules; applications can use a trust schema to constrain acceptable signing keys and guide Data validation [3]. Sync supports discovery of dataset changes so applications can retrieve available Data [4, 5]. NDN uses these primitives to discover Providers, authorize invocations, and track service execution. The detailed networking background is developed in Chapter 2.

Even when one Provider suffices, candidate discovery, authorization, selection, and result retrieval remain application responsibilities. NFN provides a named-function and

cached-computation model; NFaaS means Named Function as a Service; RICE means Remote Method Invocation in Information-Centric Networking; and NSC means Named Service Calls [6–9]. The Distributed Network Measurement Protocol (DNMP) provides a concrete example of Trust-Schema-based command authorization [10]. These NDN systems provide the computation, invocation, and authorization precedents against which this proposal examines NDNFSF’s design choices.

1.2 The Initial NDNFSF Foundation

The initial NDNFSF prototype selects one Provider from multiple candidates for each invocation. The User publishes Request Data, collects acknowledgment (ACK) Data, and publishes Selection Data for the chosen Provider, which executes and publishes Response Data. Candidates offer alternatives for the same invocation; the initial prototype does not assign them to different, interdependent roles.

Discovery and authorization also matter when only one Provider executes a request. A discovery descriptor supplies information needed to assess feasibility, such as deadlines and resource requirements; application input supplies the data needed to perform the task. In the design under evaluation, attribute-based encryption (ABE) limits descriptor access to holders of matching decryption keys. Participants check service permissions using the protected challenge values defined below (RequestNonce and ProviderChallenge), signature validation under trust rules, current permission status, and request state. Authorized but unselected candidates can read descriptors. The study also evaluates input/result protection, permission grants and withdrawals, and management costs.

1.3 Application Needs and Framework Scope

A networked application often needs more than the location of a service. A drone operator may require a camera near an area before a deadline, while an inference requester may require a particular model, sufficient accelerator memory, and an acceptable preparation delay. A

service advertisement identifies a possible source of execution, but it does not establish that the source is willing and able to undertake this task now. NDNFS separates permission to provide a service, current willingness to participate, selection for an invocation, and completion of the requested work.

These distinctions also matter within an application that usually uses one device. After choosing a drone, an operator expects subsequent camera commands to affect that drone. Substituting another available Provider would change the meaning of the command. Conversely, an image-analysis request may permit any authorized compute Provider that meets its requirements. A reusable framework must support both situations without requiring the application to implement an unrelated communication protocol for each.

The discovery descriptor contains the information needed for a participation decision. For a sensing task, this may include a coarse area, a time window, and a sensor requirement. For inference, it may include model identity, expected input size, and a deadline. Full images, detailed flight instructions, and inference prompts can be delivered after selection. The boundary follows the decision: information indispensable to assessing feasibility cannot simply be withheld until after that assessment. An application should nevertheless disclose only the necessary precision. For example, an authorized camera Provider may need an approximate search area, while an unselected candidate need not obtain the full mission route. Descriptor protection limits disclosure to authorized service Providers; it does not prevent them from learning or sharing what they legitimately decrypt.

NDNFS-UAV and NDNFS-DI expose complementary demands. The former combines device-specific operations, changing observations, live data, and physical actions. The latter combines compute selection, model preparation, large artifacts, intermediate tensors, and incremental outputs. Both require authorization and bounded request lifetimes. Their differences test whether a common invocation and named-object interface remains useful beyond one workload. Reusability is an engineering objective to be evaluated through shared mechanisms and application-specific code, rather than assumed from implementing two demonstrations.

We derive the scope by following a request from candidate discovery to execution and closure. At each stage, we identify the relationship the application needs and the failure that follows if it is not enforced.

Authorization, confidentiality, and timeliness apply across stages. Permission changes can affect an invocation throughout its lifetime and are listed separately as a cross-cutting concern.

1.3.1 Research Gap and Design Focus

NDN provides named Data retrieval and signature validation, but a Data name and a valid signature do not by themselves record or validate the service decision governing an invocation. The gap examined here is how a runtime service decision is carried into the admission and use of named Data.

Candidate discovery lets Providers report eligibility and willingness, but it does not commit an executor. A Provider's permission to offer a service authorizes that capability; it does not assign the Provider to a particular invocation or workflow role. Selection is the binding step: the initial prototype commits one Provider per invocation, while the collaboration extension commits Provider-to-role assignments and dependency edges.

For a dependent task, signature verification authenticates the producer but does not establish that the object was produced by the assigned predecessor for the active invocation or satisfies the declared dependency. The consumer must therefore check the producer, exact object name, dependency edge, and execution context.

Prior invocation and command-authorization systems address parts of this problem. NDNSF investigates a reusable protocol connecting these decisions, with authorization spanning invocation and data retrieval.

To make this gap operational, Table 1 maps each invocation stage to the relationship the framework must preserve, the failure caused by its absence, and the validation planned in Chapter 6.

This table is a requirements-to-evidence map; it does not claim that all proposed

Table 1: From invocation requirements to failure cases and proposed validation.

Stage or concern	Required relationship	Failure if missing	Proposed checks
Candidate discovery	Eligible candidates report willingness and feasibility; descriptor access is restricted	Ineligible candidate selected or descriptor disclosed to unauthorized readers	ACK eligibility and descriptor access
Selection and start	The intended Provider starts only a permitted, active invocation	Wrong device, unauthorized work, or repeated start	Permission, Selection, and replay checks
Input and intermediate Data	Correct producer, object or stream, and permitted dependency	Wrong, stale, or incomplete input advances execution	Object/stream validation; role, edge, and cache tests
Closure or replacement	Results match the active operation; closed work cannot advance its replacement	Late results revive closed work or complete the wrong operation	Deadline, cancellation, and replacement tests
Permission changes	Permission updates take effect after status validation and required key updates	A grant is unusable or withdrawn permission remains accepted	Grant/withdrawal propagation and rejection delay

collaboration checks have already been implemented or validated.

The evaluation asks whether its checks enforce the intended relationships and whether the resulting cost is justified for the studied workloads.

For each workflow in Chapters 4 and 5, validation will identify the applicable rows, test their failure cases through the corresponding framework paths, and record the outcomes. Single-Provider workflows do not require collaboration-role checks.

Chapter 6 specifies the proposed tests. Coverage is limited to the declared workflows; the exclusions below define the remaining scope boundaries.

1.4 Extending Invocation to Multi-Provider Collaboration

Some tasks require complementary services from different Providers. Independently invoking each service does not by itself specify which Provider fills each role or which intermediate objects a dependent service may accept. This proposal investigates an extension that makes those assignments and dependencies explicit while retaining the invocation foundation.

For example, nearby unmanned aerial vehicles (UAVs) could collect different views of a ground area for analysis by a compute Provider. The extension would assign sensing and processing roles. The compute Provider would check that each image comes from the UAV assigned to the required input. UAVs may need area and deadline information before deciding whether to return a positive ACK. After Selection authorizes the sensing tasks, the UAVs produce images for the assigned compute Provider. Selection supplies execution details and the required access information, not images that have yet to be captured. Descriptor encryption restricts who can read the descriptor; it does not hide names or traffic patterns. This case motivates the extension; recognition algorithms remain an application choice.

1.5 Thesis and Research Questions

The central hypothesis is that a common named-Data protocol can separate candidate discovery from authorized execution and make runtime role assignments enforceable when

intermediate objects are used. This would let UAV and inference applications share invocation and dependency-validation mechanisms while retaining their own task semantics. The study tests this hypothesis through message/state invariants, cross-application use of the same mechanisms, and matched measurements of completion and cost. It evaluates single-Provider invocation and multi-Provider collaboration separately.

- RQ1. Under what numbers of users and services, degrees of permission overlap, membership-change rates, and withdrawal frequencies does ABE-backed authorization provide a favorable confidentiality–management-cost tradeoff relative to a DNMP-inspired role-constrained authorization alternative for protected Provider discovery and invocation?
- RQ2. How can an ACK-based, single-Provider invocation be extended so that, after candidate collection closes, runtime-selected Providers receive explicit workflow roles and named dependency scopes, and accept intermediate Data—whether retrieved from a producer or a cache—only when it matches the committed producer, name, dependency edge, and execution context?
- RQ3. Under what workload, network, and failure conditions do ABE-backed discovery, role/dependency admission, and bounded recovery improve completion and latency, and when do their communication, computation, and credential-management costs outweigh the benefits?

RQ1 evaluates the invocation foundation, including a DNMP-inspired role-authorization alternative paired with content encryption and adapted to the NDNSF workflow. RQ2 investigates the collaboration extension through UAV and distributed-inference dependencies. RQ3 evaluates the utility and cost of the mechanisms studied in RQ1 and RQ2, rather than introducing another protocol; invocation and collaboration are measured separately.

1.6 Proposal Scope and Application Responsibility

The common framework scope includes discovery, permission checks, request correlation, selection, named-object exchange, and execution lifecycle. Applications define service meaning, feasibility constraints, model or sensor compatibility, and acceptable outcomes. The collaboration extension adds task roles and dependencies to that common scope. Known-Provider invocation, trusted local composition, and large-object support also matter for single-Provider tasks and are evaluated where their use is justified.

Repository support retains named objects when a workload needs persistence. Storage-system design, arbitrary workflow optimization, hard real-time flight control, safety certification, and unrestricted model partitioning are outside the research scope. Reuse will be examined by tracing the shared framework components and application-specific adapters in each workload. The two applications thus provide contrasting cases for evaluating a common service abstraction.

1.7 Expected Contributions and Current Foundation

The intended contribution is an evaluated service framework that combines discovery, authorized invocation, named-object exchange, and execution management, with an extension for complementary Provider roles. The research will determine when its authorization choices are worthwhile, how runtime assignments constrain intermediate inputs and replacement executions, and which mechanisms can be shared across UAV and inference applications. Matched workload measurements will establish benefits and limits, including conditions in which the mechanisms offer no advantage. The contribution concerns these service-level abstractions and their evaluation, not new NDN or encryption primitives.

The existing NDNSF implementation provides an invocation prototype and initial collaboration support. Its current planning path closes candidate collection before validating a plan-commit state (defined in Chapter 3), while preliminary invocation campaigns provide baseline evidence. Building on this foundation, the study will evaluate role and dependency

checks within a request, end-to-end correctness, and application benefits. Chapter 6 presents the preliminary findings and planned experiments.

Chapter 2 establishes the background and alternatives. Chapter 3 presents the framework and proposed extension. Chapters 4 and 5 describe the UAV and distributed-inference applications. Chapter 6 brings together preliminary evidence, proposed experiments, remaining work, and the timeline; Chapter 7 concludes the proposal.

Chapter 2

Background and Related Work

2.1 NDN Background and Naming Responsibilities

2.1.1 Named Retrieval, Forwarding State, and Object Identity

NDN retrieves Data by name using Interest and Data packets [1, 2]. An application expresses an Interest describing the Data it seeks. A matching Data packet can be supplied by its producer or an intermediate cache. Forwarders use forwarding information to direct Interests and pending-Interest state to deliver returning Data. This permits network-layer reuse of named Data without requiring the consumer to identify its present storage location. Availability still depends on forwarding, connectivity, and a reachable copy; a name is not a delivery guarantee.

Names have several related uses in this proposal. A service name identifies the requested operation. A message name, together with the message’s validated fields, identifies an invocation exchange. An object name identifies data such as an image, model fragment, or output tensor. Names also participate in trust rules that constrain acceptable signing keys [3]. These uses should agree, but should not be conflated: matching a service prefix does not by itself prove service permission, and a valid producer signature does not prove that the object is the input assigned to the current execution.

For example, an illustrative image name could have the form

`/org/uav7/view/task42/view2/version1`

under the sensing UAV’s producer namespace. The application must define what the task, view, and version components mean and validate the signer. A processing Provider can refer to that exact image rather than rename it as its own observation. This example explains the naming principle, not a mandatory NDNSF wire format. Names and traffic patterns can remain visible even when Content is encrypted. Sensitive identifiers should therefore not be embedded in public names merely because the associated payload is protected.

The forwarding Content Store offers opportunistic caching. A repository provides an application-managed place to retain objects for later retrieval. Neither a cache hit nor a successful digest check supplies the complete admission decision for a dependent service. The consumer still needs the producer’s authority, the intended object identity, and the current invocation or collaboration association. Likewise, a Data packet’s FreshnessPeriod is NDN metadata that guides cache-freshness treatment; it is not a service deadline or a substitute for message expiry, current permission checks, or application replay state.

Sync helps participants learn about changes in a named dataset [4, 5]. Applications can then retrieve and validate the announced Data. Thus, publication in NDN SF means making a named object available and announcing it through the configured dissemination mechanism. It should not be read as an unsolicited push of a Data packet through arbitrary forwarders. Sync recovery can repair missing announcements or trigger retrieval while time remains. It cannot ensure completion after an application deadline or during an indefinitely disconnected interval.

These distinctions define the meaning of data-driven execution here. A validated Request, ACK, Selection, result, or declared intermediate object supplies evidence for a local state transition. Timers and local scheduling policies still exist. A timeout can close collection or terminate an operation, and a planning algorithm can decide which assignment to publish. Decisions that affect other participants are communicated through named, verifiable messages; a local callback alone cannot inform a remote participant. This does not imply that all runtime state is network Data or that packet arrival alone authorizes execution.

2.1.2 From NDN Building Blocks to Service Decisions

The Forwarding Information Base (FIB) supplies name-prefix forwarding information, the Pending Interest Table (PIT) records pending retrievals, and the Content Store (CS) holds cached Data [11]. Compatible pending Interests can share a returning Data packet. This network-level aggregation is distinct from selecting several service executors. A PIT entry also has a different lifetime from a service operation: its expiration does not cancel an application handler or erase

the User’s request state.

Table 2: NDN building blocks and the service decisions that remain above them.

Building block	NDNSF use	Additional service responsibility
Interest/Data and names	Retrieve messages, results, and object segments	Define the operation, request identity, and completion condition
FIB, PIT, and CS	Forward and reuse named Data	Select work, track its lifecycle, and check acceptable object versions
Signatures and trust schema	Validate signing authority and Data integrity	Check service permission and assignment to this invocation
Dataset synchronization (generic Sync; NDN-SVS implementation)	Discover newly published message names	Validate messages and process ACK, Selection, and timeout state
Content encryption	Restrict access to protected Content	Define recipients, distribute keys, and apply permission changes

In the software stack, the local NDN forwarder handles packet forwarding, while `ndn-cxx` supplies application packet, signing, validation, and retrieval interfaces [12]. NDN State Vector Sync (NDN-SVS) supplies a concrete dataset-synchronization implementation [13]. NDNSF organizes these facilities into service interactions. Its service policies do not require forwarders to interpret GPU load, flight readiness, or a collaboration role. Those meanings belong to the framework and application contracts.

2.2 Remote Procedure Call (RPC), Robotics Middleware, and Messaging Alternatives

2.2.1 RPC Interfaces and Microservice Integration

Remote procedure call (RPC) frameworks provide typed interfaces for invoking operations on another process or host. `gRPC` is one RPC implementation that supports unary and streaming calls [14]. Robotics middleware such as `ROS` provides reusable communication abstractions for robot components [15, 16], while message-oriented middleware supports decoupled producers and consumers [17]. These systems are relevant alternatives to application-specific networking. NDNSF does not claim that endpoint discovery, asynchronous

interaction, or distributed application composition is absent from existing middleware.

Endpoint resolution identifies a destination, but does not necessarily establish that the destination is authorized and currently willing to execute a particular invocation. Middleware or the application may add those policies; this proposal treats their expression and evidence as part of the comparison.

The comparison concerns how the same application requirements are expressed. An RPC experiment must state how endpoints are resolved, connections are managed, retries are performed, and failures are detected in the selected implementation. An NDN experiment must state how names are discovered, how configured synchronization and retransmission operate, how Data is validated, and how Content Store behavior is handled. A typed call is not inherently less secure than named-object exchange; security depends on the accompanying authentication, authorization, and encryption mechanisms. Likewise, network-layer caching is useful only when object reuse is permitted by the workload and operation semantics. Reusing an image does not mean that an invocation with physical side effects may be repeated safely.

For NDNSE, the relevant question is how service decisions and application data can remain linked through names and verifiable objects. The evaluation must test that link under stated workloads rather than equate a difference in networking abstraction with superiority. The named-computation and command-authorization systems discussed below provide more direct protocol precedents.

2.2.2 Robot Components and Asynchronous Messaging

Robot software separates camera, telemetry, control, and perception components, often combining service calls with continuously updated publish–subscribe topics [15, 16]. NDNSE’s service-container design adopts this separation between component behavior and communication support. It does not replace the flight controller or imply that robotics middleware cannot implement authorization. The comparison instead asks how an incoming operation identifies the intended vehicle and how stored observations remain associated with their producer and

operation.

Message brokers provide asynchronous delivery and buffering for logs, telemetry, and workflow events [17]. The relevant design questions are where data is retained, who authorizes publication and consumption, and how delivery acknowledgment differs from task completion. A broker may be appropriate for a stable deployment. NDN’s named retrieval provides a different placement and validation model, but still requires an explicit retention policy when late retrieval matters. Neither messaging abstraction alone chooses a willing execution Provider or defines safe repetition of a physical command.

2.2.3 Microservices and Edge Computing

CFEC studies computation offloading across vehicular fog, edge, and cloud resources using mobility and resource information [18]. MIA-NDN studies microservice-aware Interest aggregation and PIT lifetime management based on input characteristics [19]. SECaaS studies the deployment of security functions at the multi-access edge, including access-control enforcement and anonymization [20]. These works widen the comparison beyond function invocation: resource placement, forwarding state, and deployable security functions are relevant parts of the surrounding system.

Differences in these systems’ objectives do not, by themselves, establish unique NDN-SF capabilities. Placement feasibility does not establish invocation permission, parameter-aware aggregation does not determine the assigned predecessor in a collaboration, and deploying an access-control service is a different question from authorizing calls to that service. Conversely, NDN-SF’s application-layer checks do not replace these systems’ resource-management or forwarding mechanisms. A claim of superiority would require a common workload and matching requirements; their published results are not interchangeable baselines for the experiments in this proposal.

2.3 Named Computation, Invocation, and Composition

Named-computation and invocation systems provide direct precedents for this proposal. NFN and NFaaS study named functions, cached computation, or execution placement [6, 7]; RICE provides an invocation handshake with authentication and security information [8]; and NSC specifies signed invocation Interests validated with an application-provided Trust Schema [9]. DNMP provides a concrete example of schema-constrained role authorization [10]. CFN and CaSCON further cover computation graphs, worker placement, or cache-assisted composition [21, 22]. Therefore, named computation, caching, workflow composition, and role-authorized calls are not claimed as isolated NDNSF novelties.

The narrower research question is how to enforce a late-bound collaboration after the positive and negative ACK collection window closes. The proposed extension assigns eligible Providers to application roles and derives predecessor and dependency checks from that assignment. A valid signature proves provenance and a request identifier provides correlation, but neither alone proves that an intermediate object was produced by the Provider assigned to the required predecessor role or is permitted on the declared dependency edge. The contribution under study is this task-specific connection between runtime assignment and Named Data admission, not a new workflow or manifest format.

The comparison will test this connection under explicit assumptions, including cached retrieval and replacement execution. It will not infer that a prior system lacks a mechanism merely because the mechanism is not summarized in a short description; if an existing system performs the same checks, the claim will be narrowed to the demonstrated integration or trade-off.

The novelty test is whether an existing mechanism already performs these checks on assigned producers, intermediate objects, and replacement executions under comparable assumptions. General workflow scheduling or waiting for dependencies is not sufficient evidence of a new protocol.

Table 3: Related precedents and the question to be tested for NDNFSF. This is an evidence map, not a performance comparison.

System	Published mechanism / scope	Question to be tested and limit
NFN/NFaaS [6, 7]	Named functions, cached computation, and placement	Check whether their execution model supplies the task-specific producer and dependency admission required here
NSC/DNMP [9, 10]	Signed invocation, Trust Schema validation, and role-constrained command authorization	Compare standing caller/role permission with a role assigned after runtime discovery
CFN/CaSCON [21, 22]	Computation graphs, worker placement, and cache-assisted composition	Compare explicit object acceptance and replacement semantics under the selected assignment

2.4 Comparing Authorization Designs under the Same Requirements

2.4.1 DNMP as a Concrete Trust-Schema Design

DNMP uses role-constrained signing to authorize measurement commands [10]. Its Network Key is the trust anchor for the role-key, device-key, and distributed-schema signing hierarchy. Given `target`, `probeType`, `probeArgs`, and the user’s `roleType`, the client finds a schema-permitted signing key among the user’s available keys. It rejects a missing match and selects the least privileged key if several match. The receiving Network Observer Daemon (NOD) dispatches a probe only after publication validation under the schema. Thus, this is command authorization, not merely verification of an arbitrary valid signature. DNMP also bounds publication lifetimes and retains recent publication state for replay protection, assuming loosely synchronized clocks.

2.4.2 Adapting the Alternative to NDNFSF

For comparison, we propose retaining the NDNFSF Request–ACK–Selection–Response workflow while replacing ABE-backed invocation checks with role-constrained signing rules. The alternative must authorize both User invocation and Provider participation, match

messages to active requests, and permit execution only after Selection. These adaptations are not features attributed to DNMP itself. Both designs require descriptor confidentiality before selection, recipient-scoped input/result protection afterward, and checks that the required service permissions have not been withdrawn.

Table 4: Alternatives under the same discovery and execution requirements.

Design	Execution decision	Non-public discovery	Management cost
DNMP-inspired alternative	Validate role-constrained signatures; check invocation state and selected Provider	Combine authorization with service-group encryption (shared-key encryption for a service-authorized group) or ABE for discovery	Role credentials, trust rules, status or expiry checks, and encryption keys
NDNSF ABE-backed	Check signatures, RequestNonce, ProviderChallenge, current permissions, and request state	Encrypt a descriptor for a service-authorized receiver set without listing current candidates	Identity credentials, ABE policies/DKEYs, status distribution, and rekeying

Role granularity is policy-defined: a role can cover multiple actions, and credentials can be reused where the rules permit. DNMP is therefore a concrete authorization precedent, not evidence that every service needs a new signing certificate.

This comparison concerns complete designs, not measured DNMP-versus-NDNSF performance. The invocation-state and encryption choices must be implemented and evaluated under matched requirements before claiming a cost advantage.

The confidentiality column requires a separate protection mechanism: Trust-Schema signing rules do not encrypt discovery descriptors. Recipient encryption protects selected exchanges; service-group encryption or ABE can protect discovery before participants are selected. Trust Schema alone does not provide confidentiality, but it can be combined with either mechanism.

The proposed alternative requires current revocation-status checks or certificate expiry. Deleting a certificate from a repository does not invalidate cached copies. We do not attribute

NDNSF's runtime revocation protocol to DNMP.

2.5 Protected Discovery and Permission Management

NDNSF uses NAC-ABE to protect Data associated with named service permissions. In the evaluated design, NAC-ABE uses a key-policy ABE (KP-ABE) construction: ciphertexts carry service attributes, while each decryption key (DKEY) carries the holder's access policy. The enhanced NAC-ABE library supplies the KP-ABE operations required by this construction [23]. This library is an implementation dependency rather than a new NDNSF protocol contribution. With this construction, the User can publish an encrypted discovery descriptor for a service-authorized receiver class without enumerating currently reachable Provider identities, while the Controller issues policy keys defining decryption access. NDNSF connects these permissions to invocation by checking protected challenge values, message signatures under trust rules, and current authorization/request state. Content encryption supplies confidentiality; the combined checks govern participation and execution. Section 3.4.1 explains the reuse of permission management and its additional costs.

The comparison must include group-key or broadcast-encryption alternatives where appropriate. A stable, small group can make conventional group encryption attractive; known Providers and simple permissions can favor signed invocation with recipient encryption. ABE becomes attractive when recipients are identified by service entitlement before their current identities are known, and multiple entitlements should remain independent of identity-signing credentials. Policy size, membership churn, encryption cost, and authority availability bound this argument.

Chapter 3

NDNSF Framework and Proposed Extensions

This chapter describes the invocation and authorization design under evaluation, including refinements to input/result confidentiality and permission updates, followed by the collaboration extension. The initial prototype supplies the single-Provider invocation foundation; the extension adds role assignments and named dependency edges. Implementation status is identified where a proposed mechanism requires further integration or testing.

3.1 Components and Application Responsibilities

The framework separates application handlers from the machinery that discovers Providers, correlates messages, and checks invocation state. A `ServiceUser` manages outgoing requests and their callbacks. A `ServiceProvider` registers service handlers and processes selected invocations. A `ServiceController` manages permissions and authorization material. These are software roles: a ground station can consume camera services while providing image analysis, and a compute node can request an upstream service while executing its own assigned work.

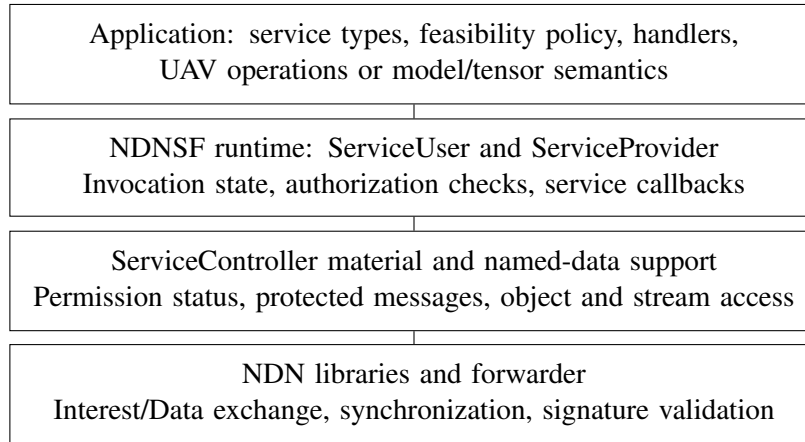


Figure 1: Responsibility groups in the framework. Lines denote software dependencies, not packet directions or a mandatory deployment topology. The Controller is a logical authority, not an on-path relay for every exchange.

Application code defines the meaning of a service and supplies its feasibility and

execution logic. The framework cannot determine from a valid ACK whether a camera truly has an unobstructed view or whether an advertised GPU can meet a prediction deadline. It can associate the ACK with the request, verify the participating identity and required protected values, and apply the configured selection policy. A positive ACK expresses willingness subject to the request's conditions; it is not evidence that execution has finished or a reservation unless the protocol explicitly provides one.

The ServiceContainer supports composition of runtime roles and local services within a process. Its local registry gives trusted components a way to invoke registered application functionality without turning every internal call into a network exchange. The application must keep the runtime and callback dependencies alive until active operations are closed. Local composition is useful for shared camera or inference facilities, but remote callers cannot select it as an alternative authorization path.

The current typed C++ interface illustrates this separation. The following shortened example uses application-defined, serializable status messages; identity provisioning, event-loop setup, and error handling are omitted. It illustrates application programming interface (API) use rather than a standalone executable.

Listing 3.1: Typed registration and invocation of a compact status service.

```
provider.addHandler<StatusQuery, StatusReply>(
    ndn::Name("/status"),
    [](const ndn::Name& requester,
        const StatusQuery& query, StatusReply& reply) {
        readApplicationStatus(query, reply);
    });

user.RequestService<StatusQuery, StatusReply>(
    candidates, ndn::Name("/status"), query,
    [](const StatusReply& reply) { displayStatus(reply); },
```

```
[ ] () { reportTimeout (); }, 2000 );
```

The handler interprets the query and fills the reply. The User interface supplies an invocation timeout and result callbacks, while NDNF manages the surrounding exchange. A service requiring confidential execution input uses the protected input-delivery path described below; this compact example does not prescribe placing arbitrary application input in discovery. Typed serialization removes some repeated application glue, but does not establish semantic compatibility between different service versions. That remains part of the service contract.

3.2 Participants, Trust, and Exclusions

The preceding component description concerns software modules. This section uses User, Provider, and Controller as protocol and trust roles: the User requests a service and selects its Provider; in the collaboration extension, it also coordinates role assignments. Providers offer and execute services. A Controller manages service permissions, key distribution, and signed authorization status. These roles are not necessarily separate devices: one node may be both a User and a Provider.

The managed organization registers identities and approves service permissions; the Controller administers that policy. Participants require a provisioned trust anchor and validation rules for identity certificates and Controller-issued Data. The User may not know which enrolled Providers will respond to a particular invocation. Forwarders and caches can deliver Data without being trusted to authorize execution. The model trusts the Controller and the User's planning decision; it does not provide consensus against a malicious coordinator.

An attacker may observe visible names and traffic, inject invalid objects, replay previously valid messages, or attempt an operation without the needed service permission. Credential compromise and intentional sharing by an authorized principal require additional controls and are not prevented by ABE alone. Billing, arbitrary business transactions, globally optimal scheduling, and protection against all denial-of-service traffic are outside the claimed

scope.

Authorization is permission to perform an operation, authentication establishes the signing identity under trust rules, and confidentiality limits who can decrypt protected content. Valid credentials do not prove honest load reports, correct inference, or truthful sensor observations. Signed status must be obtained and validated before it can affect local authorization; a disconnected participant cannot be assumed to learn revocation immediately. Deadline/expiry checks require an explicit clock policy. Tests must report the allowed skew and failure behavior rather than silently assume synchronized clocks.

3.3 Four Named Message Types

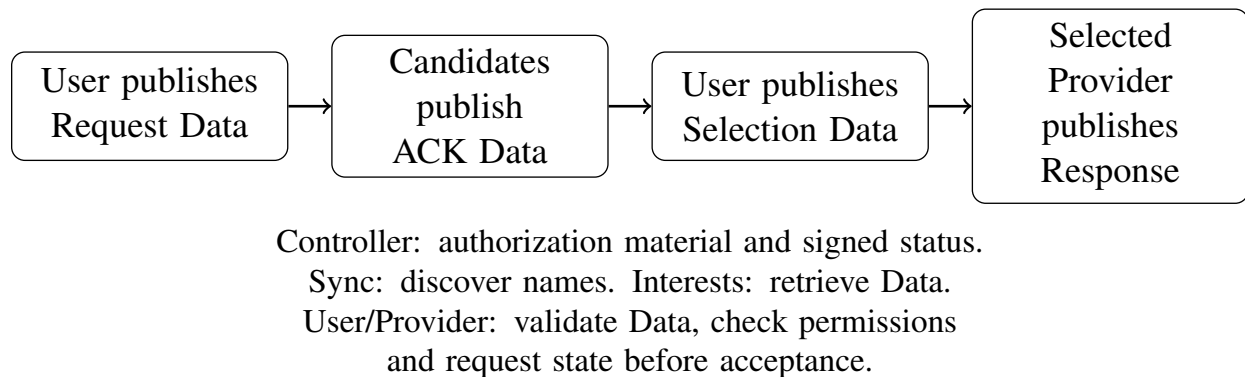


Figure 2: Logical invocation dependencies. Arrows express application causality, not unsolicited network Data delivery.

RequestNonce is a fresh User-generated application value carried in protected Request content. ProviderChallenge is a Provider-generated application value carried in protected ACK content and checked when the User constructs Selection. They are challenge values for this service exchange, not the NDN Interest Nonce used by the forwarding protocol.

Table 5: Logical message responsibilities; this is not a wire-format listing.

Data	Purpose	Principal checks
Request	Publish service requirements, minimized discovery descriptor, request identity, and RequestNonce	Validate signature and current authorization context; recover the protected descriptor before evaluating willingness
ACK	Report positive or negative participation decision; a positive ACK carries selection information and ProviderChallenge	User checks signer under trust rules, service permissions, request ID and returned RequestNonce; only eligible positive ACKs enter Provider selection
Selection	Notify the chosen Provider and supply protected input or input references and request-scoped keys	Provider validates User signature and Selection using stored ProviderChallenge; checks intended Provider, request state, and service permissions
Response	Publish completion/result metadata or references to protected results	Check authorized producer, request context, result identity, and decryption scope

The protected challenge exchange is explained below. The table describes the single-Provider invocation. Plan identity, role assignments, and dependency checks are added by the collaboration extension below, not by the initial four-message flow.

These are NDNSF Data objects. Sync supports discovery of their names, after which Interests retrieve Data, possibly from a cache. A negative ACK is an application participation decision, not a network Nack. Not every observed Request must receive a reply: invalid, unauthorized, duplicate, or expired messages are subject to validation and state checks. A positive ACK expresses willingness; it does not itself start the selected task or guarantee later resource availability.

3.4 Authorization and Confidentiality

3.4.1 *Rationale for ABE-Backed Service Authorization*

NDNSF uses ABE to protect discovery descriptors and reuses attribute-based service permissions in its invocation checks. The first choice addresses confidentiality before the participating Providers are known; the second keeps service entitlements independent of identity-signing credentials. Neither implies that ABE is always preferable to role-based authorization with suitable encryption. Withdrawal and rekeying are lifecycle costs to evaluate. Execution authorization determines whether a User may invoke a service; Provider authorization determines whether an entity may offer that service. Content confidentiality limits who can read protected content. The Controller administers these service permissions; the User and Provider check them during invocation. Signature validation under trust rules and invocation-state checks remain necessary. Permission to offer a service is distinct from selection for one particular task.

The design addresses three requirements: keep discovery descriptors non-public before participating Providers are identified; authenticate participants, verify their service permissions, and reject replayed invocation messages; and support permission grants and withdrawals during operation. Permission aggregation is an additional management benefit, not a fourth security guarantee.

Reason 1: confidential discovery before Provider selection. Before Provider selection, the User publishes a discovery descriptor containing the location, deadline, and resource requirements needed for a Provider's positive-ACK decision. For example, nearby UAVs need an observation area and arrival deadline to decide whether to participate, but do not yet need the full application input. The User encrypts this descriptor using ABE and the requested service's attribute, without enumerating Provider identities. Confidentiality is against parties lacking the required decryption capability: authorized but unselected Providers can still read the descriptor. Content encryption does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, the full application input is encrypted with a request-scoped key; Selection

protects that key for the selected Provider. The requesting User retains access. A Trust Schema alone does not encrypt content; a role-authorized design can also use ABE or service-group encryption (encryption under a shared group key) to provide discovery confidentiality.

Encrypted Request Data can be retrieved or cached without granting decryption access. This separation of delivery from disclosure also holds for other content-encryption schemes; it is not unique to ABE.

Reason 2: reusing attribute-based permissions for service invocation. Confidential discovery alone does not justify ABE-backed execution authorization: ABE encryption could instead be combined with role-based invocation checks. NDNSF chooses to administer both Provider and User service permissions through the existing attribute-authority infrastructure, separately from identity-signing credentials.

Provider permissions determine access to RequestNonce; User permissions determine access to ProviderChallenge. NDNSF checks RequestNonce in ACK Data and validates Selection Data using the stored ProviderChallenge. Signature, current-permission, and request-state checks remain separate requirements (Section 3.4.2). Decryption is therefore one part of the invocation check, not permission to execute any request.

DNMP instead uses role signing keys and a Trust Schema to constrain command targets and actions [10]. In the DNMP-inspired alternative (Section 2.4.2), retaining ABE for discovery would require managing both decryption permissions and role-based invocation permissions. NDNSF uses the attribute-authority infrastructure for both service-level checks, without adding role signing credentials solely to represent service membership. Trust rules for validating Data signatures remain necessary.

The evaluation will measure the costs and benefits of this shared management infrastructure. Provider and User permissions remain distinct; User permission decryption keys and challenge processing add costs beyond descriptor encryption. For policies requiring command-argument restrictions, DNMP's schema-based checks may be a better fit than

NDNSF’s service-level checks. The evaluation must compare both designs at the same policy granularity, including credential updates and invocation overhead.

Reason 3: service-permission aggregation as an additional management benefit.

KP-ABE associates attributes with ciphertext and an access policy with each decryption key (DKEY) [23, 24]. NDNSF uses /SERVICE/<service> for Provider access and /PERMISSION/<service> for User access. Within one authority and parameter generation, an entity’s service permissions can be combined in one individually issued DKEY. For example, one User’s policy can allow detection OR mapping, while another allows detection only. Their identity-signing credentials need not change to represent these different service sets; each service retains its own ciphertext attribute.

Compared with DNMP’s role-key-based authorization [10], NDNSF represents service entitlements in the DKEY rather than in additional role-specific signing credentials. Where equivalent entitlements require several distinct role credentials, this avoids issuing and maintaining those additional signing credentials. The comparison depends on role granularity: one DNMP role can permit several commands, and Trust Schemas can support other credential arrangements. The evaluation will therefore measure independently managed credentials alongside DKEY size, policy complexity, distribution, rekeying, and runtime costs.

3.4.2 Authorization Mechanisms, Lifecycle, and Costs

Identity authentication. The User and Provider validate Data signatures and certificate chains under the applicable trust rules. This identifies an accepted signer; it does not alone establish that signer’s permission to offer or invoke the requested service.

Service-permission verification. In the ABE-backed invocation path, the User generates a fresh random value, *RequestNonce*, and includes it in ABE-protected Request content. This is an NDNSF application-level value, not the NDN Interest Nonce used for forwarding duplicate and loop handling. A Provider that recovers it returns it in ACK Data; the User compares the

returned value with the one stored for the active request. Thus RequestNonce supports both request binding and a check of access to the service-protected value. The Provider generates *ProviderChallenge* and includes it in ABE-protected ACK content. The User uses the recovered ProviderChallenge to construct Selection Data. Before execution, the Provider validates Selection Data using ProviderChallenge stored for that request. Together with current service-permission checks, these checks determine whether the Provider may offer the service and whether the User may invoke it.

These checks demonstrate access to protected challenge values, not the creation of new permissions or a formal proof of exclusive key possession. Authorized key or challenge sharing is outside the claimed threat protection. This reuses attribute-based permission management rather than a separate set of role-based invocation rules. It is not cost-free: confidential discovery already needs Provider DKEYs, whereas User permission DKEYs and challenge processing are additional authorization costs. ABE confidentiality combined with DNMP-inspired role authorization remains a valid alternative.

Replay protection. Participants match messages to an active request and its protocol state. The Provider records whether ProviderChallenge has already been used to accept Selection Data for execution; receiving the same Selection again must not start another execution. This requires appropriate retention and invalidation of request state, including across restart. Neither encryption nor possession of a challenge value alone prevents replay. These admission checks do not establish exactly-once application side effects across crashes.

Runtime permission grants and withdrawals. *PolicyStatusData* denotes a Controller-signed named Data payload, not a grant by itself. It identifies the service and current ControllerVersion, validity interval, policy digest, optional ABE public-parameter name and digest, the Controller certificate, and revocation targets. Participants validate the enclosing Data signature and this status before updating local permission or DKEY state; the status distributes and authenticates policy state but does not independently grant permission.

The Controller updates permission status for both operations. An ordinary grant updates the target entity's complete permission policy; that entity retrieves a replacement DKEY without changing the ABE public parameters. Withdrawal additionally rotates the Controller's ABE master key and public parameters. Entities retaining permissions obtain replacement DKEYs encoding their remaining permissions. The master key stays with the Controller. Rotation is Controller-wide, not confined to the withdrawn service.

In the proposed update-discovery path, which is not yet fully verified end to end, the Controller publishes updated public parameters as signed named Data and uses Sync to announce their availability. Participants retrieve and validate the parameters and Controller-signed `PolicyStatusData`, then refresh their DKEYs as required. Ordinary grants also require participants to discover and retrieve updated permission status, even though the public parameters have not changed. Sync is an update-discovery mechanism, not the authority for granting permission. The local implementation supports status retrieval and parameter rotation; the dedicated Sync announcement path remains to be verified end to end.

Current-permission checks enforce withdrawal after the updated status has been retrieved and validated. Parameter rotation restricts exchanges encrypted under the new generation. Disconnected nodes cannot apply updates immediately, and previously disclosed keys or plaintext cannot be taken back.

Lifecycle comparison and evaluation. DNMP specifies validity bounds and replay-state handling [10]; these do not by themselves provide immediate withdrawal of an unexpired role key. For the comparison, a DNMP-inspired design can revoke the relevant credential and require execution nodes to check signed revocation status or reject expired certificates. The execution node then rejects the revoked or expired credential; distributed certificate copies need not be retrieved. Both approaches require update propagation, and neither retracts previously disclosed keys or plaintext.

Focused integration tests exercise NDNSF's runtime revocation checks, including

identity-, certificate-, and service-scoped withdrawal. These tests establish behavior at the exercised runtime boundaries, not network-wide instantaneous revocation or lower cost than certificate-based alternatives. The remaining evaluation should measure status-propagation delay, rejection delay, rekeying traffic, and effects on unaffected members. Runtime status validation and key rotation support runtime permission changes, with explicit propagation and rekeying costs. They are not ABE-exclusive capabilities. NDNSF introduces ABE for confidential discovery and reuses its service-permission management for invocation checks and permission updates; signatures and request-state checks remain necessary, separate parts of the design.

Discovery confidentiality applies to descriptor Content against parties lacking the required ABE decryption capability. It does not conceal the descriptor from other authorized candidates or provide traffic-analysis resistance. In the request-scoped protection design, the User encrypts application input with a symmetric key and delivers the necessary key material through Selection, encrypted for the chosen Provider's authenticated public key. The response key is shared by the requesting User and selected Provider. Other Users cannot decrypt the protected result under the key-secrecy assumptions; the Provider necessarily knows the plaintext it produces. These are the protection requirements for the evaluated design, rather than claims about every historical implementation path.

3.5 Candidate Information, Selection, and Admission

Selection is a policy decision over eligible candidates, not a substitute for permission checks. The current User implementation exposes first-positive-ACK, random-positive-ACK, all-positive-ACK, and application-supplied selection policies. Selecting several Providers for independent executions does not, by itself, assign complementary roles or enforce dependencies. The proposed collaboration extension adds those semantics; they should not be attributed to the initial single-Provider invocation workflow.

Early ACK arrival is an operational selection rule, not proof of the lowest execution latency. Random choice can serve as a controlled policy baseline; an application policy can

Table 6: Information needed by selection and its limits. Application-specific fields are service contracts, not mandatory fields in every ACK.

Information	Decision it supports	Interpretation limit
Request/service identity and deadline	Associate a candidate with the requested operation	A matching identifier does not establish permission
Positive/negative ACK and reason	Distinguish willingness from explicit refusal	No ACK may instead reflect loss or delay
Service-specific capability and resource report	Filter unsupported models, sensors, memory or timing constraints	Signed reporting proves provenance, not the truth of a capacity prediction
Available artifacts or compatible state	Estimate preparation cost before selecting a Provider	Reuse still needs version and authorization checks
Provider identity and protected challenge information	Validate the participant and selected exchange	Neither guarantees future availability

consider estimated completion time or preparation cost. Comparisons should hold the collection window and eligibility checks fixed when isolating the policy’s effect. Collaboration must account for dependencies and compatible assignments, not merely choose the individually fastest candidate for every role.

Selection identifies the intended executor. Execution-time admission determines whether that Provider can still accept the work under its current resource limits. The lifecycle and failure outcomes below distinguish refusal, queued work, execution, and completion.

3.6 Invocation Modes and Execution Lifecycle

3.6.1 *Discovery, Known Providers, and Local Composition*

Normal discovery is appropriate when the executor has not been chosen. The User chooses among positive ACKs that pass validation and policy checks. Selection timing follows the configured policy rather than one universal waiting rule.

A supported first-positive-ACK path may select before timeout. A collection-based policy

waits until its declared closure condition is met. The proposed snapshot-based collaboration commits only after its candidate collection closes.

A negative ACK communicates non-participation; silence cannot distinguish refusal from loss or disconnection. Selecting several independent executions does not assign complementary roles and dependencies as a collaboration plan does.

Known-Provider invocation is appropriate when the operation must use a particular device, for example, stopping the camera that was previously started. The current `RequestServiceTargeted` path can reuse previously bootstrapped one-time authorization material to avoid repeated ACK/Selection exchanges under its supported conditions. It still checks current authorization and request state. The fast path is not universal: request-scoped confidentiality needs a Selection envelope to carry the input and response keys, and the current implementation uses the bootstrap/ACK/Selection path for that case. The explicitly Selection-gated input mode is rejected by the Selection-free Targeted interface. Thus, targeting a known Provider is an application requirement, whereas omitting exchanges is a conditional implementation optimization.

Trusted local composition operates within a process through the local registry. It has neither remote discovery nor the same network threat boundary. Evaluation should identify whether a handler was reached locally, by normal discovery, or by a targeted exchange. Otherwise an apparent invocation improvement might merely measure a different operation.

3.6.2 Admission, Progress, and Failure

An API submission, admission to a runtime queue, acceptance by a Provider, and completion of application work are different events. Under overload, reporting latency only for completed requests can hide dropped or rejected work. The runtime therefore needs observable outcomes for refusal, timeout, and terminal results, while an evaluation must count all submitted requests in its stated denominator. Selection-status queries can help determine the observed status of selected work; they are not proof of a physical effect or a distributed transaction.

Applications must also distinguish cancellation of interest in a result from reversal of an action. A recording can be stopped, but previously recorded frames still exist. An inference worker can stop producing new output, but a completed remote action cannot be undone by deleting requester state. Retrying requires the application's stated idempotency or compensation policy. For the proposed collaboration extension, replacement additionally requires old assignments and late callbacks to stop advancing the current plan.

3.7 Control Messages, Streams, and Complete Objects

Small service messages communicate intent, permission evidence, and status. Large application data should not be copied into each control message. An object reference allows a service to identify a model fragment, recording, or image, while the recipient retrieves its named Data separately. The reference and service contract must supply enough information to select the intended version and validate the reconstructed object. The concrete interface determines the available name, size, digest, segmentation, and protection metadata; not every object type needs an identical collection of fields.

The important separation is between identifying an object, obtaining its bytes, and authorizing its use. A digest can detect a mismatch with the expected bytes only when the expected digest itself is trusted. A signature binds Data to a key under the application's trust rules. The invocation or collaboration checks establish whether that producer's object is permitted as an input to the current execution. Recipient-scoped key delivery controls access to protected Content. None of these checks should silently substitute for another.

An ongoing stream has a different lifecycle from a complete object. Live video may prefer recent frames and tolerate explicitly identified gaps. A model artifact generally requires every needed byte before loading. Incremental inference requires ordered application events and a terminal outcome, but its token stream is not necessarily interchangeable with a camera stream interface. NDNFSF supplies named-data mechanisms; applications define acceptable gaps, buffering limits, and completeness rules. The evaluation must measure the chosen policy,

including its discarded data and queueing costs.

Named-object reuse is valuable in the multi-view case: the compute Provider returns annotations and references to the original images, and provides the requester with the image content keys it is permitted to receive. The requester can retrieve those images from available producers or caches and validate their original signatures. This avoids producing a second image object merely to change the receiver. It does not guarantee that a cached copy remains available; retention requirements need an explicit producer or repository policy.

3.8 Proposed Extension: From ACKs to a Collaboration Plan

For this proposal, a candidate snapshot is the canonical, ordered set of validated eligible Provider offers observed when the ACK collection window closes. Each record includes the Provider identity and the signed offer/ACK information used for eligibility, such as its service capability and validity data. The snapshot digest is a cryptographic digest of that canonical encoding. A logical request is identified by its request ID; an execution attempt ID distinguishes one run of that request, including a replacement. The execution identity is the request/attempt pair. The snapshot is immutable for that attempt; a replacement receives a new attempt and must not silently reuse the old assignment.

The current NDNFSF implementation already closes candidate collection, checks the assigned snapshot digest, verifies that selected Providers belong to that snapshot, and rejects conflicting plan commits. These mechanisms provide an implementation basis for the collaboration study. The remaining research evaluates dependency acceptance, recovery, and cross-application behavior against the acceptance rules defined below.

The extension changes the selection problem from choosing one executor to assigning complementary task roles. The application supplies a role/dependency template, required services, feasibility constraints, and deadline. The template may be written manually; automatic generation is not necessary to establish the framework contribution. A replaceable planning policy selects among eligible positive ACKs after the collection window closes. Closing the

window freezes the candidates observed for that decision, not all possible Providers in the network. ACK claims can become stale, so a selected Provider still performs execution-time admission and may reject work it can no longer execute.

In the proposed design, the User signs a Plan Manifest containing the request identity, candidate-snapshot digest, exact role-to-Provider assignments, permitted named dependency edges, authorization/key scope, and execution identity. Its plan digest is computed over a canonical plan body that excludes the digest field itself. Providers validate that signed plan before deriving local dependency checks; a long-term certificate does not specify this task's assignments. The User must record the committed plan and reject a different plan for the same execution identity. This is a local state-machine rule under the trusted-User model, not a consequence of signing alone or protection against a Byzantine User signing two plans.

The technical difficulty is that the abstract dependency graph can be known before discovery, but its concrete signing identities and object namespaces cannot yet be fixed. The proposed protocol therefore separates an application-supplied template from its post-ACK binding. Each consumer derives its permitted predecessor, object-name constraints, and key scope from the same committed assignment. A later ACK cannot change only one consumer's interpretation. This is stronger than waiting until an arbitrary input is available, but does not establish optimal placement or agreement among malicious planners.

3.9 Proposed Dependency Admission and Execution State

Under the proposed acceptance rules, a downstream Provider checks that the execution is active and within its deadline before accepting a dependency. The Data signature must validate under the applicable trust rules. Newly produced inputs must match the assigned request, plan, execution identity, predecessor, and declared edge; historical inputs instead require the explicit reuse authorization described below. Name, format, and key-scope constraints still apply to both. Required segments must validate, and incomplete or malformed objects cannot satisfy an edge. The same checks apply whether Data is retrieved from a producer, a cache, or a repository. The

plan references objects in their actual producers’ authorized namespaces rather than republishing them under a different producer identity.

Table 7: Proposed task-state contract and required negative checks.

State/event	Permitted transition	Required rejection or bounded behavior
Collecting ACKs	Close window and validate candidate set	Late ACK does not silently alter a committed plan
Selected	Validate Selection and perform local admission	Positive ACK alone, wrong Provider, or already-used ProviderChallenge cannot start work
Waiting for dependencies	Execute when required authorized edges are satisfied	Wrong role, undeclared edge, incomplete object, or wrong active execution is rejected
Completed/cancelled	Return recorded status under duplicate-message policy	Duplicate publication does not implicitly repeat a side effect
Failure/reassignment	Explicitly terminate or authorize a replacement execution	Old execution must not acquire authority from the new assignment; allowed result reuse is explicit

The proposed recovery rules separate retrieving an object again from repeating computation. Once a Provider applies a replacement, old execution messages and callbacks cannot advance the new execution.

An existing artifact is reusable only when the new signed plan explicitly references its exact immutable identity, original producer and provenance, destination dependency, and permitted key scope. The artifact retains its original name and signature; it is not relabeled as newly produced. The consumer checks that explicit reuse authorization instead of requiring the historical artifact to claim the new execution identity. All other old objects remain ineligible.

Application checks must additionally establish that a captured image is still timely or that a tensor/state object is compatible. Thus, an old artifact may remain useful without allowing messages from its old execution to advance the replacement. Application-defined compensation does not provide distributed atomic transactions or exactly-once physical action.

RQ1 evaluates the authorization design and its alternatives. RQ2 tests the plan and

state rules through wrong-role and wrong-edge inputs, replay, cancellation, and equivalent validation of Data obtained from producers and caches. These collaboration experiments extend the invocation evidence presented in Chapter 6.

3.10 Conditional Correctness Argument

The preceding admission argument is conditional on uncompromised signing keys, correctly enforced trust and authorization checks, and Providers that obey the specified state machine. To permit execution only after Selection, every path that starts a handler must check an active assignment and confirm that the one-time authorization state has not already been used to start execution.

To enforce the declared dependencies, every input path, including retrieval from a cache, must check the dependency edge and the assigned producer. Historical artifacts instead require explicit reuse permission. The artifact reference must identify exact Data, for example by a full name including its implicit digest or a validated segment manifest with content digests.

After replacement, old workers and late callbacks must not advance the new execution. Restart recovery must preserve or safely invalidate the relevant one-time state. A valid signature and cache FreshnessPeriod alone cannot establish these properties.

The study will test these conditional invariants by injecting a wrong-role object within the same request, an undeclared input from a correctly signed producer, and a late callback after replacement. Each case isolates a property that a request ID alone cannot express. The argument concerns protocol acceptance and execution state under the stated assumptions; application-result correctness and physical side effects require separate validation.

Chapter 4

UAV Application over NDN

NDN-UAV exercises service discovery, authorization, data delivery, and execution management through control, telemetry, video, recordings, and collaborative sensing. Single-Provider operations examine the invocation foundation; multi-view collection additionally tests complementary roles. Chapter 6 specifies the comparisons and measurements.

4.1 Application Structure and Service Workflows

The UAV application connects an operator's ground station with drone-side service containers. Drone services expose camera operations, telemetry, and mission-related operations through NDN. The ground station can also provide an object-detection service, so the same application includes both service consumption and service provision. Application code interprets camera, vehicle, and mission data; NDN supplies the reusable invocation and data-delivery mechanisms.

These operations cover different framework requirements. Selecting a currently available sensing Provider exercises discovery and ACK-based selection. Addressing an already chosen vehicle exercises invocation associated with that Provider rather than selecting an arbitrary substitute. Telemetry and mission-status retrieval exercise the association of returned Data with the intended vehicle and active operation. Long-running operations supply cases for cancellation, deadlines, and handling late status. Tests must distinguish rejection of an invalid command from the flight controller's physical execution of an accepted command.

Live video and recordings exercise different data paths. Camera Start/Stop is a service-control operation; the resulting live stream uses the named-data streaming interface. Recording references and complete stored objects require exact object retrieval rather than assumptions about cache persistence. These functions therefore contribute to validating stream lifecycle, ordering and gap handling, protected content delivery, and data-product retrieval.

Video codecs, image processing, operator interfaces, and flight safety remain application responsibilities; their use of NDNDF is nevertheless part of framework validation.

4.2 Operator Workflow and State Interpretation

A typical session starts with an operator discovering sensing services and choosing a vehicle. Subsequent camera or mission operations retain that vehicle identity unless the operator or a stated mission policy authorizes replacement. The interface associates status with the vehicle and operation that produced it. A delayed observation must not be presented as evidence that a newly issued command succeeded. Observation time, operation identity, and the application's stale-data threshold are therefore part of the validation contract.

The current UAV experiment assumes a camera fixed to the vehicle body. This is an experimental configuration that simplifies geometric calibration; it is not a requirement imposed by NDNDF. Vehicle-attitude disturbance and camera-to-body calibration therefore remain application-level validation concerns.

The proposed evaluation follows a complete session rather than testing isolated buttons: obtain status, start a camera stream, inspect incremental frames, stop the stream, find a recording, and retrieve the selected recorded object. Camera control uses service invocation; frame delivery and recording retrieval use their respective data paths. An implementation may record while no live viewer is attached, so recording availability should not be inferred from stream subscription. The test records whether the configured application supports that mode and whether recording references identify the correct producer and version.

4.3 Live Video, Recording, and Perception Services

Live viewing and recorded retrieval have different objectives. The live path prioritizes recent usable frames within a latency budget; delayed frames may no longer be useful to the operator. A recording path instead needs a defined retention interval and identifiable segments for later retrieval. Stream/session identity, producer identity, and frame or segment sequence

Table 8: UAV workflow contracts to exercise in application validation.

Operation	Application meaning	Observable framework outcome
Telemetry and status	Observation of the selected vehicle	Correct producer and operation association; stale data identified
Camera Start/Stop	Change the selected camera’s activity	Authorized invocation and terminal status distinct from frame arrival
Live video	Recent frames from the requested stream	Start delay, useful frame delivery, gaps, bounded buffering
Recording retrieval	Obtain a particular stored capture	Exact object and producer validated; Content access enforced
Mission operation	Issue bounded work and follow progress	Admission, progress, timeout, and completion distinguished

must prevent a late frame from an earlier session from appearing in the current one. These are application validation requirements, not claims that every delivery policy is implemented.

The test should vary frame size, production rate, available bandwidth, and receiver buffering independently. Reducing resolution or dropping obsolete frames may reduce viewing delay but also reduce visual information; it is not an unconditional networking improvement. Report delivered frame rate, age of displayed frames, gaps, and recording completeness alongside service-call latency. Authorization to start recording, access a stream, and read a retained object must be checked at the corresponding operation or object boundary.

Perception adds computation to acquisition: a sensing service supplies a named image, an analysis service obtains the permitted image and runs a fixed detector, and the requester retrieves annotations referring to that image. Telemetry, camera control, recording, flight commands, and analysis therefore exercise different framework paths. Multi-view collaboration combines those paths rather than replacing the rest of the UAV application.

4.4 Flight Commands and Local Safety

Three responsibilities must remain distinct: the NDNSF runtime checks service authorization and request state; the UAV application manages operator authority and mission state; the vehicle and its safety logic determine whether an operation is safe.

The evaluation will examine where vehicle-specific operator authority is checked, in addition to service-invocation permission. The current ground station checks operator leases; the drone-side execution handler does not independently repeat that check. End-to-end tests will assess whether expired or unauthorized operator commands are rejected along the command path.

Flight-related services connect the application to the vehicle's flight-control interface, such as MAVLink and PX4 [25, 26]. Authorization to submit a command is separate from its operational safety. Geofencing, vehicle readiness, collision avoidance, and local failsafe behavior belong to the vehicle/application safety design. The networking study should use bounded supervisory commands and an explicit loss-of-contact policy, rather than assume that a service invocation is suitable for every low-level flight-control loop.

For a bounded navigation or camera-pointing operation, validation should record vehicle readiness, command acceptance, observed progress, and the final application outcome separately. Readiness checks include a current autopilot heartbeat, a suitable position estimate, battery condition, and the permitted flight mode. The flight controller's command acknowledgment and observed vehicle state must be distinguished from NDNSF ACK Data. A positive NDNSF ACK cannot establish that the vehicle reached a waypoint. Late status from an old command must not complete a newer one.

Timeout and reconnection tests must verify a locally configured hold, return, or landing policy appropriate to the vehicle and operating conditions. Continuous manual control requires a separate control-rate and stale-command safety assessment; it is not assumed safe merely because supervisory calls work. In the fixed-camera configuration, interpreting a view also requires time-aligned vehicle attitude and camera-to-body calibration. Service-message delivery alone does not establish flight safety or geometric accuracy.

4.5 Mission Assignment and Partial Failure

Mission progress, reassignment policy, and compensation remain application responsibilities. The NDNDF runtime carries the associated service exchanges; it does not interpret a vehicle's mission outcome or provide atomic rollback of physical actions.

Multi-vehicle patrol provides a secondary workload: an application divides a region into tasks, tracks their completion, and decides whether unfinished tasks can be reassigned. Its relevance is the association of task state and returned observations with the active assignment. This is broader than multi-view analysis, but it should not become an additional unbounded algorithm-development requirement. The primary collaboration experiment below fixes its task and analysis method so that network and protocol effects can be isolated.

For both workflows, failure experiments should include a missing observation, a disconnected selected UAV, and delayed status after a replacement decision. Report the time during which a feasible Provider is actually reachable as well as the request deadline. Retransmission and synchronization cannot manufacture radio coverage or extend an expired application time budget. A partial completion rate can therefore reflect availability limits even when retrieval recovery behaves correctly.

4.6 Representative Collaboration: Multi-View Collection and Analysis

An operator chooses a target location or area on a map. Request Data carries only the service-protected descriptor needed to assess coverage, viewing feasibility, sensors, and timing. Eligible UAVs return positive ACKs. The proposed workflow assigns complementary viewing roles and a processing role, then delivers execution details and appropriate keys through Selection.

Each sensing UAV signs encrypted ViewImage Data under its own authorized namespace, with observation time and available pose/calibration metadata. The compute Provider retrieves permitted images, performs the specified analysis, and publishes signed Annotation Data. Its encrypted result supplies original image references and permitted image keys to the requester,

which retrieves the images and renders annotations. The workflow reuses the original signed images rather than republishing them under another producer's identity.

The Global Positioning System (GPS) alone does not establish camera visibility, depth, or correspondence between views. Improved recognition or localization requires ground truth, a single-view control, and a specified fusion method. MiniNDN [27] and PX4 Software in the Loop (SITL) [26] can exercise network behavior and application/autopilot integration respectively; neither establishes real-flight safety.

Chapter 5

Distributed Inference over NDNF

NDNF-DI is the distributed-inference (DI) application over NDNF. It exercises discovery, authorization, model and tensor exchange, and execution management through single-Provider inference and collaborative model execution. Its computation and retained state provide a contrasting workload to UAV sensing and external actions. Chapter 6 evaluates the shared framework mechanisms and application-specific checks.

5.1 Application Structure and Inference Workflows

NDNF-DI provides an inference application layer over the framework. It interprets model and task descriptions, coordinates model preparation and execution, and exposes inference results to callers. Its native requester and Provider interfaces include request submission, service registration, result/event handling, and cancellation; conversation-related interfaces also carry continuation state. The presence of an interface is not evidence that every model or recovery configuration has passed end-to-end tests.

Python bindings adapt the native C++ path rather than maintain a second request state machine.

The NDNF runtime supplies reusable operation, completion, and event-reading mechanisms. DI defines model preparation, placement, tensor contracts, K/V compatibility, and the rules for committing a completed turn to conversation state.

A single-Provider inference request exercises service discovery, permission checks, selection, protected input delivery, and result retrieval without requiring model partitioning. Preparing a cold model and reusing a resident model exercise different readiness and artifact-access paths. Model files and recorded artifacts may use managed repository storage; the forwarding Content Store remains opportunistic caching, not guaranteed persistence. DI defines model identity, compatibility, and placement policy, while NDNF supplies service lifecycle and

named-object exchange.

Streamed inference and conversation workflows extend coverage to incremental outputs, cancellation, terminal status, and request-specific state. Model weights, activation tensors, and key/value (K/V) state have distinct lifetimes. The model computes K/V tensors and the inference runtime retains them for later decoding; shared buffers can avoid copying the whole cache at each step [28]. Resident weights do not replace this state. DI owns tokenization and KV compatibility rules; framework validation concerns the authorized request, data exchange, and lifecycle through which the application uses that state. The evaluation must separate the benefits of local K/V reuse from those of NDNSF’s communication and execution mechanisms.

5.2 Representative Collaboration: Multi-Provider Inference

A planner uses the prepared model and validated ACK offers to choose a partition and Provider assignment after candidate collection closes. Selected Providers prepare or reuse their assigned resources after authenticating Selection. Pipeline partitioning assigns sequential layers or layer groups. Tensor partitioning divides operations or tensors and requires the corresponding collective exchanges. Providers exchange activations or collective inputs over declared dependencies. Open Neural Network Exchange (ONNX) [29] describes the computation graph; export alone does not implement cross-Provider collectives or establish numerical correctness. Model partitioning has substantial prior work [30,31]; the NDNSF question concerns runtime assignment and authorized named-data execution.

Autoregressive generation also uses each generated token in a later decoding step. This temporal dependency differs from forward computation through the layers. The evaluation separates model preparation, prefill, time to first token, subsequent decoding, state storage, and data transfer. Claims remain specific to tested models, precisions, partitions, and devices. A feasible single-Provider baseline is retained; if a model cannot fit one device, distributed feasibility is meaningful but a speedup against a non-running baseline is not.

5.3 From a Model Graph to Runtime Assignments

Local preparation verifies a reusable model package, its source identity, and supported interfaces. In this chapter, `PreparedModel` denotes that locally validated, reusable model package and session state; it is not a Provider assignment, resource reservation, or proof that a remote runner is ready.

After ACK collection closes, the native planner validates offers, enumerates supported partitions, and validates placement. A reusable or manually authored template may constrain that search; it does not preselect the runtime Providers.

The planner obtains artifact references and seals the execution plan before committing it through the NDNFS runtime. After authenticating Selection, each selected Provider obtains, assembles, or reuses its permitted resources before execution.

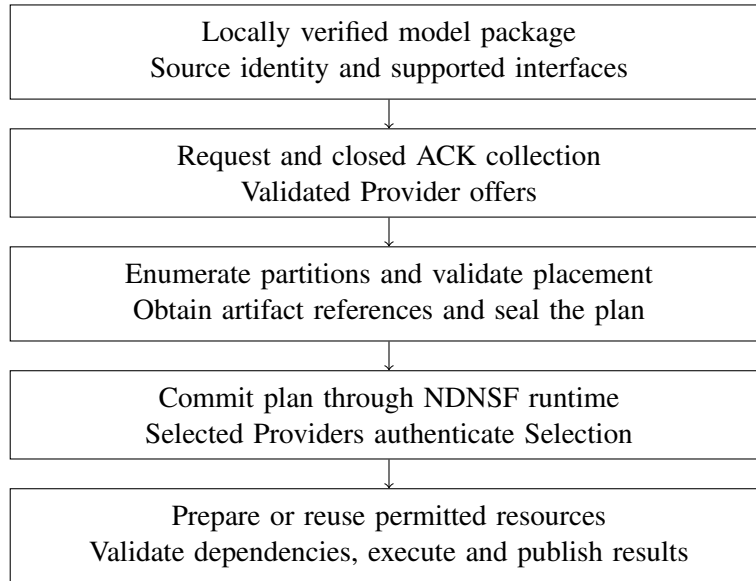


Figure 3: Model preparation and request-specific planning in the native path. Arrows show processing order; model, device, and recovery coverage still require separate validation.

Each fragment contract needs defined input and output tensor names, shapes or supported dynamic-shape constraints, element types, serialization, and model-version compatibility. A split across a skip connection or a branching graph may have multiple live outputs, not a single vector. The receiving fragment must obtain all required inputs before execution. A useful negative test

supplies a correctly signed tensor with the wrong shape or model identity: protocol provenance may be valid while the application contract is not.

Pipeline partitioning assigns consecutive computation stages and transfers their boundary tensors. Tensor parallelism splits an operation across participants and adds exchanges such as reductions or gathers. Model replication assigns independent work to complete copies. These are different execution strategies. Early exit changes how much model computation is performed [32]; combining predictions from several models changes the analysis method. Both must be separated from placement effects. Edge–cloud describes device placement, not a distinct partition operator. ONNX export supplies a graph representation [29]; usable cuts and collective operations still depend on the supported runtime and model operators.

Preparation cost matters when a Provider lacks the required artifact or compatible loaded session. Evaluation should distinguish artifact transfer, session creation, accelerator memory allocation, and execution. A warm run that reuses resident weights cannot be compared to a cold baseline without reporting that difference. Placement can favor a Provider that already holds compatible state, but that policy must respect request authorization and cannot assume that cached state is transferable to any new request.

5.4 Planning Inputs, Feasibility, and Policy Comparisons

The planning study considers a bounded set of supported partition candidates. It separates feasibility constraints from optimization preferences: an unsupported operator, an incompatible tensor contract, or insufficient memory excludes an assignment; estimated execution or transfer time ranks otherwise feasible assignments. Estimates need a stated source and measurement age. A Provider’s positive ACK is a willingness report, not an independently verified hardware benchmark.

Compare a fixed valid mapping, a simple resource-aware mapping, and any implemented adaptive policy on the same Provider candidates and supported partitions. Record planning time, per-Provider utilization, and end-to-end cost. Where a policy predicts completion time,

Table 9: Inputs to the proposed DI planning comparison. These are evaluation dimensions, not a claim of a completed general optimizer.

Planning input	Feasibility / cost question	Experimental control
Model and partition contract	Operators, precision, tensor interfaces and compatible artifacts	Freeze model/export identity and supported cuts
Device resources	Weights, activations, workspace and retained key/value (K/V) state fit	Measure peak memory; do not use weight size alone
Computation and communication	Do capacity or execution-time benefits justify transfer costs?	Profile boundary bytes and per-stage time under the same load
Preparation and residency	Are artifacts, sessions and compatible state already present?	Separate cold preparation from warm execution
Permission and availability	Can each assigned role obtain its required objects before the deadline?	Apply authorization and deadline constraints before ranking

compare its prediction with the observed value; a policy using only feasibility rules needs no timing-prediction claim. If no complete feasible assignment exists, report that outcome rather than silently dropping a dependency. The initial question is whether runtime information improves assignment under stated conditions. Model splitting, Provider selection, and data-dependency enforcement remain independently testable.

5.5 Tensor Exchange and Autoregressive State

For a pipeline, an upstream fragment produces an activation tensor, makes it available as protected named Data, and supplies its reference through the declared dependency. The downstream fragment retrieves and validates that object, checks the tensor contract, and performs its assigned computation. A fan-out reuses an upstream output for several authorized consumers. A fan-in waits for the set of inputs required by the computation. The collaboration protocol decides which producers and edges are permitted; DI decides whether the tensor values and layouts are suitable for the model.

Autoregressive inference adds a dependency across decoding steps. When K/V caching is enabled, computation after prefill uses the preceding token and retained K/V state instead of recomputing the full prefix. With layer-based partitioning, each participating runtime can retain the K/V tensors for its assigned layers. A generated token or the information needed to select it returns to the component driving the next decoding step. Consequently, the forward path within one evaluation can follow a directed acyclic graph while the whole generation process repeatedly traverses that graph over time.

Keeping weights in accelerator memory does not eliminate the logical K/V inputs. A runtime may reuse device buffers and bind them directly rather than serialize and transmit the entire cache on each step [28]. Reassignment or conversation continuation needs an explicit compatibility and state-transfer policy; silently starting a replacement with unrelated K/V state is not valid recovery. The evaluation separates savings from local K/V reuse, remote transfer costs, and framework orchestration costs.

The initial numerical validation should compare a supported partitioned execution with an unsplit execution of the same model, precision, inputs, and decoding policy. Appropriate tensor tolerances or deterministic output comparisons must be specified before measurement. A small synthetic model can validate a tensor route, but cannot establish memory feasibility, throughput, or recovery for a large pretrained model. These controls make it possible to distinguish a working communication path from a correct and useful distributed-inference application.

We are preparing multi-node GPU experiments on TigerCluster, the external multi-GPU cluster used for this evaluation, to evaluate NDNFS-DI. Providers will run on separate GPU nodes to check inference correctness and measure end-to-end latency, inter-Provider data-transfer overhead, and peak GPU memory use. These cluster experiments will complement, rather than replace, UAV wireless and mobility tests.

Chapter 6

Research and Evaluation Plan

6.1 Cross-Application Validation

The study uses UAV and DI workflows to examine which discovery, authorization, dependency, and lifecycle rules can be shared, and which decisions require application-specific contracts. End-to-end measurements follow each workload through its invocation and named-object exchange paths. Single-Provider operations examine RQ1 and RQ3; workflows with complementary roles also examine RQ2.

The proposed division of responsibilities is explicit: the framework checks who may produce an object and where it may be used; the application checks whether its content is useful and correct. Table 10 makes that boundary testable. The NDNFSF runtime may invoke application validators through a common interface; this does not make their domain knowledge an NDNFSF protocol rule.

Table 10: Proposed shared checks and application responsibilities, to be tested in both applications.

Shared framework check	UAV contract	DI contract
Validate signer, assigned role, and dependency edge	Define required view, observation time and admissible sensor metadata	Define model/partition identity and tensor dtype/shape compatibility
Validate complete named objects and explicit reuse permission	Decide whether an existing image still satisfies coverage and timing	Decide whether artifacts or key/value (K/V) state match the model and continuation
Enforce local cancellation/replacement state before dependent work	Stop or compensate physical actions; report what actually occurred	Drain computation, release resources, and commit or discard continuation state
Authenticate provenance, not semantic truth	Validate visibility, localization and recognition against observations	Validate numerical outputs against a reference execution

For example, a correctly signed image from the assigned UAV can still show the wrong area. A correctly signed tensor can still contain wrong values. The framework must not certify either as a correct application result. The research deliverable is a shared set of object-acceptance

and execution-state rules, with explicit points for application checks. Both applications will test invalid cases and confirm that valid work still completes. Control, telemetry, streaming, recording, preparation, and single-Provider inference remain part of this study; collaboration adds tests rather than replacing that coverage.

Table 11 groups the planned application experiments by the framework mechanisms they exercise.

Table 11: Application workflows and intended NDNFSF validation coverage.

Application workflow	Framework mechanism	Required observation
UAV control/telemetry; DI single-Provider inference	Discovery, selection, service authorization	Correct Provider/request association; unauthorized invocation rejected
UAV live video; DI incremental output	Stream or invocation-output lifecycle	Validated outputs, termination, cancellation, and defined gap handling
UAV recordings; DI model/input objects	Named-object references and retrieval	Exact permitted object reconstructed; tampered or wrong object rejected
UAV mission status; DI preparation/conversation	Operation and request lifecycle	Correct status ordering; late actions cannot revive closed work
UAV multi-view; DI partitioned inference	Role assignment and named dependencies	Assigned producers only; missing dependencies and replacement handled
Permission changes in either application	Controller status and invocation checks	Grant discovered/applied; withdrawn permission rejected after update

For each evaluated workflow, the study will document the application function, NDNFSF API path, configuration, expected outcome, and recorded results. Local tests will check individual components; multi-process experiments will evaluate communication and execution across nodes. Image-analysis accuracy, numerical correctness, and vehicle behavior will be assessed with application-specific criteria alongside service completion.

The current native implementation has results from tests of individual components and selected multi-process workflows. The planned campaigns will extend this evidence to the

selected model, device, and recovery configurations. New tensor/hybrid, multi-accelerator, and large-language-model recovery configurations require their own experiments; the historical small-model measurements provide a separate starting point. Representative workflows will cover the mechanism categories in Table 11, with conclusions limited to the configurations evaluated.

6.2 Preliminary Evidence and Its Limits

Invocation campaigns below measure discovery, selection, and execution under their recorded configurations; they do not test a multi-role collaboration plan. The earlier DI diagnostic is separate application evidence, not proof that the initial framework supported role-aware collaboration.

The following numbers are retained from recorded campaigns, not new experiments in this revision [33]. Comparative uncertainty is estimated across runs with independent seeds, not by treating requests within one run as independent observations. Here, CI denotes confidence interval. Latency calculated only from successful requests must be reported alongside completion and timeout rates. A baseline label identifies the tested implementation and configuration, not every feature of the named framework.

In the work-efficiency comparison, NSC-4 also completed 1,800/1,800 requests, with 1.002 execution starts per request (rounded). This does not establish a work-efficiency advantage over NSC. The mobility comparison uses common traces, publication times, deadlines, and four Providers per system. The switching holdout uses seeds 72–81 and a registered reachability-based classification. Of its 1,312 aligned requests per system, NDNSF completed 1,311, gRPC 1,297, and NSC 1,296. For each seed, p95 is the 95th-percentile latency calculated from that system’s successful requests, so the latency comparison is affected by different failure sets. Overall coverage controls using seeds 62–71 are a separate campaign, not the parent sample of this holdout. The small-model diagnostic is retained from earlier records; its raw-run/model provenance remains to be rechecked. Communication/setup overhead is a possible explanation,

Table 12: Selected evidence and limits; values are unchanged from their records.

Campaign	Observation	Supported interpretation / limit
Ten-seed switching holdout	1,312 matched switching requests; NDNF minus gRPC seed-p95 difference: -597 ms, 95% CI $[-992, -200]$; versus NSC: -2618 ms, CI $[-2916, -2315]$	Lower conditional tail latency in the tested four-Provider configuration, not universal mobility or success-rate superiority
Six-seed work-efficiency repeat	NDNF: 1,798/1,800 completed, 1,800 execution starts; gRPC-PAR-4: 1,800/1,800, 7,180 starts	Fewer starts than this parallel policy; starts are not messages, bytes, CPU time, or energy
Earlier small-model DI diagnostic	Ten successful runs/layout: mean 259.4 ms distributed versus 186.8 ms single Provider; p95 283.9 versus 206.3 ms	Splitting was slower for this model/configuration; small success-conditioned sample does not establish a general tail-latency result

not an established cause.

The transition experiment reports 357/357 completions for NDNF without pre-registering Provider addresses, 0/357 for static-list baselines, and 357/357 for pre-registered baseline variants. It isolates the discovery configuration. A gRPC resolver or other discovery integration can update a candidate list [34]; the result does not prove that gRPC or NSC cannot discover a new Provider.

Historical v0.1 loss results include 599/600 completions at 1% per-link loss, 598/600 at 3%, and a best recorded 539/600 at 10%. The last is a selected historical run, not a current aggregate estimate or a matched comparison against later baseline data. The expression $1 - (1 - \ell)^4$ assumes four independent packet traversals with equal loss ℓ ; it is not the full service-failure probability after retries. Timeouts under heavy loss do not, without further interventions, identify one exclusive cause or rule out protocol/policy effects.

Prototype admission, ACK-policy, and availability observations are preserved in Appendix A. Their raw-run provenance or comparison controls require further verification, so they motivate experiments rather than establish the present comparative conclusions.

6.3 Evidence Needed for Each Research Question

Table 13: Planned comparisons, correctness tests, and measurements for each research question.

RQ	Comparison	Correctness / failure observation	Costs and outcomes
RQ1	ABE-backed authorization; DNMP-inspired role rules plus content encryption (Section 2.4.2); suitable group-key combinations	Unauthorized call never starts handler; replay, wrong challenge, grants, withdrawn/expired permissions, status delay	Completion, discovery delay, crypto work, credential/policy size, grant and withdrawal traffic
RQ2	Same task graph with static/dynamic assignment; controlled removal of each admission check	Wrong role, edge, name, key or execution; same checks for producer/cache Data; incomplete objects; cancellation or reassignment	End-to-end completion, validation cost, intermediate bytes, redundant work, bounded recovery
RQ3	Matched Provider counts, tasks, capacities, deadlines and failure schedules; feasible single-device control	Include failed and timed-out requests; distinguish network, admission, execution and result-retrieval failures	Completion and latency together; cold/warm costs; per-run uncertainty; conditions with no benefit

Each campaign will record source, dependency, and model versions, configurations, seeds, measurement intervals, raw results, and analysis scripts. Scaling experiments will vary candidates and services for discovery/authorization, and dependency edges and object sizes for collaboration. Ablations will disable individual checks only in controlled tests to isolate their effects. Failed attempts will be retained and classified by their first observed failure; runs that fail before request measurement begins will be reported separately from measured invocation outcomes.

6.4 Application Experiments and Interpretation

Individual invocations or application sessions supply observations. Comparative uncertainty is estimated across independently repeated runs, with shared seeds or traces paired across systems; requests within one run are not assumed independent.

Before each comparative campaign, the workload definition will specify the primary outcome, completion deadline, and criteria for a practically useful improvement. Pilot runs will inform repetition counts; these choices will be fixed before the comparison and reported alongside effect estimates and uncertainty.

To operationalize RQ1, the study defines the following authorization cost model before selecting comparison configurations.

The first RQ1 comparison will retain ABE-encrypted discovery in both designs and change the invocation-authorization mechanism: protected challenges versus DNMP-inspired role-constrained signing. Both retain signature validation, Selection, request-state checks, current service-permission checks, and recipient-scoped input/result protection. This isolates authorization choice from discovery encryption; it does not measure the original DNMP implementation. Service-group encryption is a separate sensitivity comparison. Initial authorization tests use one User, four candidate Providers, and one permitted and one unpermitted service before scaling the configuration.

The matched alternatives will use the same permission matrix, discovery protection, and status-validity bound. Shared discovery costs remain in the total but are separated from incremental invocation-authorization costs.

Credential and policy model. Consider one Controller domain, one ABE key generation, and n authorized Users. Let k and r denote the mean service permissions and role certificates per User. Each User retains one identity-signing certificate.

The role alternative adds nr role certificates; ABE-backed authorization adds n policy DKEYs. Provider population and permissions remain fixed, and both designs retain the Provider DKEYs needed for discovery.

Per-service roles ($r = k$) add $O(nk)$ certificates; one composite role per User adds $O(n)$. With fixed k , both grow linearly with n . These counts describe the chosen credential arrangements, not limits imposed by Trust Schema or DNMP.

Certificate counts are not private-key or NDN packet counts. Shared roles share

permission definitions, not private keys. Policy information remains: explicit DKEY service lists contain nk entries in total.

We will measure key sizes, validation rules, and distribution traffic alongside object counts. NAC motivates this distinction [35]; its CP-ABE analysis does not establish NDNF’s KP-ABE authorization cost.

Lifecycle cost. For design X and additive metric d , define the cost per submitted logical invocation as

$$\overline{C}_{X,d}(T) = \frac{C_{X,d}^{\text{setup}} + \sum_{p \in \mathcal{P}} C_{X,d}^p(T)}{Q(T)}, \quad \mathcal{P} = \{\text{invoke, grant, withdraw, maint}\}. \quad (1)$$

Here $Q(T) > 0$ counts logical invocations submitted during a common interval T , not retries. Each C^p is the total cost of that activity during the same interval; maintenance includes background status checks and renewals, without double counting.

CPU time, transmitted bytes, and management operations are measured separately. Setup and recurring costs are also reported separately; completion, update delay, latency, and memory are not combined into this additive cost.

An ordinary NDNF grant replaces the target’s DKEY without rotating public parameters. Controller-wide withdrawal rotation can require replacement DKEYs for all remaining authorized entities, including Providers.

The adapted role baseline uses signed status, version checks, and validity intervals. Withdrawing a User’s invocation permission need not require reissuing every role certificate. This lifecycle is a baseline extension, not a claim about DNMP.

Both designs reject known status rollback and stop admitting calls when required status expires. They use the same validity bound and clock-skew assumption; disconnected nodes may lose availability.

NDN `FreshnessPeriod` and `MustBeFresh` do not establish authorization validity [2]. Withdrawing a Provider’s permission to read discovery descriptors requires

discovery-key updates in both designs; neither can recall disclosed keys or plaintext.

Table 14: Planned RQ1 configurations. Counts are analytical examples, not measurements.

Configuration	Controlled example	Question to test
G1: Per-service roles	100 Users; four disjoint five-service bundles, 25 Users per bundle. One certificate per allowed service: 500 role certificates versus 100 User DKEYs.	Do fewer objects reduce total cost after policy size and cryptographic work are included?
G2: Shared combination roles	Same permissions; one bundle-role certificate per User: 100 role certificates versus 100 DKEYs.	Does ABE retain an advantage when roles already aggregate permissions?
G3: Frequent withdrawals	Repeat G1/G2 with matched grants and withdrawals. Fix invocation rate and vary withdrawal rate; test User invocation and Provider discovery-read withdrawal separately.	When does rekeying outweigh provisioning savings?

The 100-User example counts Users issued credentials, not Users required to invoke services concurrently. Paired sweeps will vary population and permission overlap. Role-update rules, workloads, cache conditions, and repetitions will be fixed before comparison.

Updates preserve exact permissions. We will compare costs, legitimate-call completion, and rejection of expired or rollback status, withdrawn permissions, and replayed calls. The aim is a conditional tradeoff, not a universal ranking.

For UAV orchestration, the control relays the same signed, encrypted image objects through the User without changing their producer identity; direct exchange uses the assigned producer names. Both use equivalent protection and the same processing code. Image inputs are held fixed for this comparison, so it does not test a recognition benefit from additional views. Numerical tolerances for DI will be declared for the chosen precision before runs, not adjusted to fit the observed errors.

Each run records its workload, software revision, and network configuration. Report submitted, admitted, selected, completed, rejected, timed-out, and cancelled work separately where observable, with explicit denominators and deadlines.

Latency distributions identify the included requests and are reported alongside failure rates. A single exploratory run is labeled as such and does not supply a between-run uncertainty estimate.

For UAV sessions, record command completion latency, video start delay, useful frame delivery, recording retrieval completion, and status age at the operator. For multi-view work, additionally record the number of usable views, processing completion, and the analysis metric against labeled observations. The framework experiment first holds the recognition algorithm fixed. A separate application study is needed before attributing higher recognition accuracy to additional views.

For DI, report model preparation, prefill, time to first token, subsequent output rate, total completion, peak device memory, and bytes transferred between Providers. Compare cold and warm conditions separately. Evaluate a feasible single-Provider case alongside a supported partitioned case. If the distributed deployment exists because the model cannot fit one device, report that capacity benefit directly rather than inventing a speedup over an infeasible execution. Profile communication, serialization, cryptography, and model computation to locate the limiting cost.

Authorization evaluation should exercise both directions of service permission, not only successful decryption. Negative cases include an unpermitted caller, an unpermitted Provider, a validly signed message with the wrong protected value, and reuse after request state is closed. Grant experiments measure how an enrolled entity learns and applies a new permission. Withdrawal experiments measure update propagation, status validation, parameter/DKEY refresh, and rejection after the update is applied. Disconnected nodes and old ciphertext are separate cases with the limitations stated in Section 3.4.1.

The collaboration ablations retain request-ID isolation and signature validation. Removing assigned-producer checks tests a wrong-role object within the same request; removing edge checks tests an undeclared input from an otherwise authorized participant. Replacement tests inject old work after reassignment. Outcomes record the check responsible for rejection

and whether prohibited execution occurred. If another retained check rejects the object, that experiment does not establish the removed check's independent necessity. Valid-input controls prevent rejection of all work from being mistaken for correct enforcement.

Framework reuse can be assessed by documenting which invocation, authorization, object-transfer, and lifecycle components are shared by UAV and DI, and which adapters remain application-specific. Source size alone is not a sufficient maintainability metric. Record the changes needed to add a service type and the contracts that must be supplied. This supports the framework's engineering rationale while leaving protocol correctness and performance to their own evidence.

For the boundary study in Table 10, both applications will exercise the same NDNSE runtime paths for wrong-producer rejection, undeclared dependencies, and late work after cancellation. Separate inputs will pass cryptographic and dependency checks but violate an application contract: an out-of-window image or a tensor with an incompatible declared model/shape. Record which layer rejects each case and verify that rejected input cannot start dependent application work. Valid controls must still complete. A correctly shaped tensor with incorrect values illustrates the remaining limit: numerical validation against a reference, not a signature, is needed. Record the adapters and runtime changes needed for each workload; duplicated checks or new application-specific branches in the runtime would indicate that the proposed division of responsibilities needs revision. These observations support a reusable contract for the tested workloads, not universal applicability to all services.

Deployment records should include dependency versions, model hashes, topology, link parameters, clocks, retry policies, cache configuration, and key-management settings without publishing secrets. Separate application failures from startup, configuration, and collection failures. A failed setup is not evidence of a protocol's inability to complete a request. These records allow the final dissertation to distinguish supported claims, useful negative results, and configurations that remain untested.

6.5 Primary Research Scope and Feasibility

The initial UAV graph will assign two image-collection roles and one analysis role; its matched control uses the same images and recognition algorithm. The initial DI case will target a two-Provider YOLO (You Only Look Once) partition, with every tensor crossing the partition boundary declared as a dependency. Model version, cut, precision, input set, and numerical tolerance will be fixed after graph-compatibility checks and before comparative runs. A feasible unsplit execution supplies the numerical reference and performance control. These are planned configurations, not completed validations.

The minimum scope includes valid completion, invalid-input rejection, and one controlled cancellation/replacement case in each graph. Additional partition strategies, LLM streaming, larger deployments, and real-flight trials are extensions, not prerequisites for these bounded framework findings. Existing control, telemetry, video, and single-Provider inference workflows retain their validation roles. If either core graph cannot be validated, its RQ2 claim remains unsupported and the scope or schedule must be reconsidered with the committee. The analysis will report both beneficial and unfavorable operating conditions.

Validation proceeds from local correctness tests to controlled MiniNDN campaigns, UAV simulation and bench integration, and supported physical devices. Each stage establishes evidence only for the tested environment. Physical-flight experiments require the appropriate approval; simulation alone cannot establish real-flight safety or perception performance.

6.6 Existing Foundation and Remaining Research Work

The invocation protocol, application prototypes, candidate-closure and plan-commit checks, and preliminary campaigns provide the existing foundation. Remaining work comprises the complete authorization alternative, integrated dependency/recovery checks, and the matched application experiments described above. The dissertation will connect each RQ answer to its reproducible evidence and state the resulting limitations.

Work will proceed from a tested end-to-end workflow and matched control to additional

scale or model diversity where these help answer a research question. Correctness tests and reproducible measurements take priority over optional features. If a model partition or flight platform is unavailable, the evaluation scope will be adjusted and the resulting limits reported.

6.7 Timeline and Decision Points

Table 15: Revised internal targets; defense is conditional on evidence and committee approval.

Period	Research milestone / decision
September 2026	Agree on three RQs and bounded claims; freeze supported application graphs and implementation/evidence baseline; resolve correctness blockers
October 2026	Run predefined correctness tests and matched comparisons; reduce optional scope if a required end-to-end test has not passed
November 2026	Freeze accepted results and provenance; write RQ answers, costs, and limitations; keep missing results visibly unverified
December 2026	Deliver an integrated dissertation draft to the advisor; prioritize analysis and revision over optional features
January 2027	Advisor/committee review, revisions, defense readiness decision, and confirmation of current university submission/notice requirements
February 2027	Target final dissertation defense after required reviews and evidence closure; allow time for substantive corrections
Spring 2027	Complete approved corrections and final submission by the applicable Graduate School deadlines; confirm exact dates separately

The February 2027 defense target depends on evidence and committee approval. CS guidance calls for an advisor-approved complete dissertation to reach committee members two weeks before the defense [36]. The Graduate School lists February 5, 2027 for the graduation application and doctoral candidacy form, and March 26, 2027 for the defended final ProQuest

submission [37]; these dates must be rechecked before filing. If core end-to-end evidence is missing at the January review, scope or timing will be reconsidered with the committee.

6.8 Risks, Mitigation, and Limits

ABE adds authority, computation, and rekeying costs; disconnected revocation has propagation limits. Candidate ACKs do not prove future capacity or honest reporting. Cache availability does not guarantee task completion. Tests confined to one process do not establish distributed failure behavior. UAV safety and perception accuracy need evidence beyond service-message delivery. Findings from selected hardware and models must be interpreted within those configurations.

Table 16: Risks, detection evidence, and scope-preserving responses.

Risk	How it becomes visible	Response
Dependency or deployment mismatch	A binary, model, credential, or runtime configuration differs across hosts	Freeze versions and artifact identities; verify the deployment before measuring a request chain
Integration delays	End-to-end trace fails before a validated terminal result	Freeze one supported graph; separate provisioning, protocol, and application failures before adding features
Communication overhead outweighs execution benefits	Boundary transfer and setup offset any parallel-execution benefit	Report the crossover or negative result; do not hide it by changing precision or baselines
Delayed permission withdrawal	Disconnected participants retain stale status or old keys	Measure propagation and rekeying; enforce declared status validity; exclude retroactive secrecy claims
Unavailable hardware or flight approval	Required device, operating condition or approval cannot be obtained	Use declared simulation/bench scope; retain real-hardware questions as limitations
Unreproducible or confounded results	Missing provenance, unequal candidates, or inconsistent failure denominators	Preserve records, repair controls and repeat only the affected claim; withhold unsupported conclusions

Chapter 7

Conclusion

This dissertation studies a reusable service framework over NDN. It builds on NDN's initial ability to discover multiple candidates and select one Provider for an invocation, evaluates its invocation and data-exchange mechanisms, and extends them to complementary Provider roles and named dependencies.

RQ1 will identify conditions favoring ABE permission reuse or complete role-authorized alternatives as permission overlap, membership, and withdrawal frequency change. RQ2 will determine how assignments made after ACK collection constrain new and explicitly reused named objects without allowing old execution messages to advance their replacements. RQ3 will measure benefits and costs across UAV and DI workflows, and test which checks the framework can share and which require application-specific validation.

These are proposed design and protocol contributions, not a new encryption primitive or the first use of named computation. Existing campaigns establish limited invocation findings; the collaboration and cross-application claims still require their declared end-to-end evidence. The proposal asks the committee to assess the research scope, comparisons, validation criteria, and feasibility. The dissertation will report supported answers and limitations, including conditions in which the proposed mechanisms offer no advantage.

Appendix A

Preliminary Prototype Observations and Evidence Limits

This appendix records preliminary observations from prototype records that informed this proposal [38]. It is a provenance ledger, not a second results section: the observations are not newly reproduced, and the raw traces, denominators, implementation identity, workload, and measurement definitions must be recovered before a final claim relies on them.

Table 17: Preliminary prototype observations retained for traceability and their permitted interpretation.

Observation	Prototype record	Permitted use and limit
Load/admission at 100 requests/s	p95 4,254.39 ms and 3.55% timeout without admission; about 81 admitted requests/s, p95 268.12 ms, and 0% timeout with admission. Denominators and accepted/rejected work are unverified.	Motivates separate load, accepted-work, latency, and timeout reporting; does not establish equal-work improvement.
Historical ACK suppression	2,209 selections/2,250 requests and 1.82% success for an overloaded-nearest-Provider policy; 100% for a suppression variant.	Motivates a declared positive/negative ACK experiment; not evidence about the current ACK protocol or general authorization.
Two-Provider availability	74.62% completion at 10 requests/s and 73.73% at 30 requests/s; coverage intervals, deadlines, and failure causes are unrecovered.	Motivates reachability and failure-category measurements; does not establish universal mobility or retry benefit.
Unequal-provider comparisons	Archived tables compare single-Provider baselines with multi-Provider NDNSF configurations.	Motivates the matched Provider-count and failure-schedule campaigns in Chapter 6; cannot isolate a framework effect.

bottomrule

The ledger preserves favorable and unfavorable observations without upgrading them to current evidence. The planned evaluation will either reproduce each entry under controlled

conditions or cite it only as motivation for controlled work.

References

- [1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, B. Zhang, Named data networking, in: ACM SIGCOMM Computer Communication Review, 2014, pp. 66–73.
- [2] Named Data Networking Project, NDN Packet Format Specification: Data Packet, <https://docs.named-data.net/NDN-packet-spec/current/data.html>, accessed September 9, 2026.
- [3] Y. Yu, A. Afanasyev, D. Clark, k. claffy, V. Jacobson, L. Zhang, Schematizing trust in named data networking, in: Proceedings of the 2nd ACM Conference on Information-Centric Networking, 2015, pp. 177–186. doi:10.1145/2810156.2810170.
- [4] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.
- [5] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.
- [6] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.
- [7] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.
- [8] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.
- [9] D. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, Tech. Rep. NDN-0074, Revision 1, UCLA Computer Science Department (2021).
URL <https://named-data.net/wp-content/uploads/2021/05/ndn-tr-0074-1-nsc.pdf>
- [10] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122. doi:10.1145/3357150.3357397.
- [11] C. Yi, A. Afanasyev, I. Moiseenko, L. Wang, B. Zhang, L. Zhang, A case for stateful forwarding plane, Computer Communications 36 (7) (2013) 779–791.

- [12] ndn-cxx: Ndn c++ library with experimental extensions, <https://github.com/named-data/ndn-cxx>, , accessed 2026.
- [13] Statevectorsync: A library for distributed real-time applications over ndn, <https://github.com/named-data/ndn-svs>, , accessed 2026.
- [14] grpc: A high performance, open source universal rpc framework, <https://grpc.io/>, , accessed 2026.
- [15] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Ros: an open-source robot operating system, ICRA Workshop on Open Source Software.
- [16] Ros 2 documentation, <https://docs.ros.org/>, , accessed 2026.
- [17] V. John, X. Liu, A survey of distributed message broker queues, arXiv preprint arXiv:1704.00411.
- [18] M. S. ud din, M. A. ur Rehman, B.-S. Kim, Cfec: An ultra-low latency microservices-based in-network computing framework for information-centric iovs, *IEEE Transactions on Services Computing* 17 (6) (2024) 3199–3212.
- [19] M. Imran, et al., Mia-ndn: Microservice-centric interest aggregation in named data networking, *Sensors* 23 (3) (2023) 1411.
- [20] R. Tourani, et al., Towards security-as-a-service in multi-access edge, in: *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*, 2019, pp. 358–363.
- [21] M. Król, S. Mastorakis, D. Oran, D. Kutscher, Compute first networking: Distributed computing meets ICN, in: *Proceedings of the 6th ACM Conference on Information-Centric Networking*, 2019, pp. 67–77. doi:10.1145/3357150.3357395.
- [22] C. Barrios, E. Alker, M. Kumar, CaSCON: Cache-assisted service composition orchestration leveraging NDN, in: *2025 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2025, pp. 122–129. doi:10.1109/SMARTCOMP65954.2025.00061.
- [23] S. Dulal, T. Yu, S. Liu, A. R. Thieme, L. Zhang, L. Wang, Enhancing NAC-ABE to support access control for mHealth applications and beyond, arXiv:2311.07299 (2023). doi:10.48550/arXiv.2311.07299.
URL <https://arxiv.org/abs/2311.07299>
- [24] V. Goyal, O. Pandey, A. Sahai, B. Waters, Attribute-based encryption for fine-grained access control of encrypted data, in: *Proceedings of the 13th ACM Conference on Computer and Communications Security*, 2006, pp. 89–98.
- [25] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, M. Khalgui, Micro air vehicle link (mavlink) in a nutshell: A survey, *IEEE Access* 7 (2019) 87658–87680. doi:10.1109/ACCESS.2019.2924410.

- [26] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.
- [27] Minindn: A mininet-based ndn emulator, <https://github.com/named-data/mini-ndn>, , accessed 2026.
- [28] ONNX Runtime Authors, How to configure the past present share buffer, <https://onnxruntime.ai/docs/genai/howto/past-present-share-buffer.html>, , accessed September 11, 2026.
- [29] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.
- [30] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697.3037698.
- [31] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017.226.
- [32] S. Teerapittayanon, B. McDanel, H. T. Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: 2016 23rd International Conference on Pattern Recognition, 2016, pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.
- [33] NDNSF Project, Proposal research revision: Claim and evidence audit, Companion repository document: docs/PAPER/proposal-defense/research-revision-audit.md, , september 9, 2026; includes frozen campaign paths, evidence limits, and unresolved research gates.
- [34] gRPC Authors, Custom name resolution, <https://grpc.io/docs/guides/custom-name-resolution/>, , accessed September 9, 2026.
- [35] Named Data Networking Project, NAC: Automating access control via named data, <https://named-data.net/publications/zhang2018nac/> (2018).
- [36] University of Memphis, Department of Computer Science, Information for computer science ph.d. students, https://www.memphis.edu/cs/pdfs/forms_grad_phd_programinfo.pdf, , accessed September 9, 2026.
- [37] University of Memphis Graduate School, Graduate school graduation information, https://www.memphis.edu/gradschool/current_students/graduation.php, , spring 2027 deadlines, checked September 16, 2026.
- [38] T. Ma, NDNSF: A data-centric service framework for secure, collaborative, and dynamic networked applications, , source manuscript for the prototype observations, University

of Memphis, Tables 5–6 and Section 5.6. Historical reported observations, not newly reproduced measurements (Jun. 2026).